

Anno Accademico 2025/2026

Leonardo Donati

Appunti
Reti e Calcolatori

Teoria e esercizi.



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Indice

1	Introduzione ai Protocolli	4
1.1	Comunicazione fra host	4
1.2	Metodologia per comunicare	4
1.3	Protocolli	5
1.3.1	Protocol Data Unit (PDU)	5
2	Standard Suite	6
2.1	Modello ISO-OSI	6
2.1.1	Livello Fisico	7
2.1.2	Livello Data-Link	7
2.1.3	Livello Rete	7
2.1.4	Livello Trasporto	7
2.1.5	Livello Sessione	7
2.1.6	Livello Presentazione	7
2.1.7	Livello Applicazione	7
2.1.8	PDU dello stack ISO-OSI	7
2.2	Modello TCP/IP	8
2.2.1	Livello Host-to-Network	8
2.2.2	Livello Rete	8
2.2.3	Livello Trasporto	8
2.2.4	Livello Applicazione	8
2.3	Circuit switching	8
2.4	Packet Switching	8
3	Livello Host-to-Network	9
3.1	Definizione e tecnologie	9
3.2	Ethernet	9
3.2.1	Frame Ethernet	10
3.2.2	Preamble	10
3.2.3	Indirizzo di Destinazione e di Origine	10
3.2.4	Tipo	10
3.2.5	Data e Dimensione	10
3.2.6	CRC (Cyclic Redundancy Check)	10
3.3	Protocollo ARP	11
3.3.1	ARP Cache	11
3.3.2	RARP	11
3.4	Interconnessione fra LAN	12
3.4.1	Hub	12
3.4.2	Bridge	12
3.4.3	Hub VS Bridge	12
3.4.4	Switch	12
3.4.5	Bridge VS Switch	12
3.4.6	Bridge in Linux	13
3.5	Virtual LAN	14
3.5.1	Tagged VLAN	14

4	Livello IP	15
4.1	Importanza e compiti del livello Network	15
4.2	Protocollo IP	16
4.2.1	Datagram	16
4.3	Ip Addressing	17
4.3.1	Classe di Indirizzi IP	17
4.4	Subnetting e Supernetting	17
4.4.1	Subnetting	18
4.5	Architettura di Internet	18
4.5.1	Autonomous System	18
4.6	Routing	19
4.6.1	Teoria del Routing	20
4.6.2	Distan Vector Protocol	21
4.6.3	Problemi	22
4.6.4	Algoritmi Link State	22
4.6.5	Differenze fra Distan Vector Protocol e Algoritmi Link State	23
4.7	Routing a scala geografica	24
4.7.1	Border Gateway Protocol (BGP)	24
4.7.2	Open Short Path First (OSPF)	25
4.7.3	Struttura di un messaggio OSPF	25
4.7.4	Struttura della gerarchia OSPF	26
4.8	IPv6	26
5	Livello Trasporto	27
5.1	Introduzione	27
5.1.1	Multiplexing e Demultiplexing	27
5.2	Protocollo UDP	28
5.2.1	Campo Checksum	28
5.3	Protocollo TCP	29
5.3.1	Affidabilità	29
5.3.2	Trasferimento con buffer	29
5.4	TCP Segment	30
5.5	Inizializzare e chiudere una connessione TCP	31
5.6	Affidabilità nel protocollo TCP	32
5.6.1	Stima del Timeout	32
5.7	Gestione del Buffer TCP	33
5.7.1	Sliding Windows	34
5.7.2	Flow Control	34
5.7.3	Congestion Control	35
5.7.4	Buffer Bloating	37
6	Socket Programming	38
6.1	API (Application Program Interface)	38
6.1.1	Socket	38
6.2	Creare e gestire una connessione	38
6.2.1	Codice C Server Side:	41
6.2.2	Codice C Client Side:	42
6.2.3	Codice Python Server Side:	43
6.2.4	Codice Python Client Side:	43
7	DNS (Domain Name System)	44
7.1	Definizione:	44
7.2	Host identifiers	44
7.3	Name Servers	45
7.3.1	Resource Records	45
7.3.2	Risoluzione dei nomi in modo distribuito	46
7.4	Pacchetto DNS	46
8	Esercizi	47
8.1	Creare un rete	47

Capitolo 1

Introduzione ai Protocolli

1.1 Comunicazione fra host

Obiettivo: trasferire un insieme di bit da un host a un altro.

Bisogna anche assicurare:

- Massima velocità (**Performance**).
- Nessun fallimento o malfunzionamento. (**Affidabilità**).
- **Sicurezza** nella comunicazione.

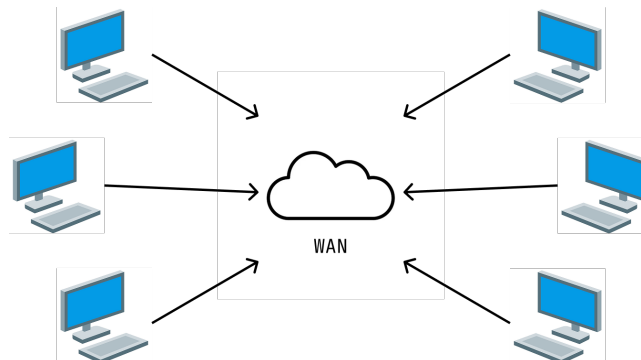


Figura 1.1: Comunicazione fra Host

1.2 Metodologia per comunicare

La comunicazione, da molto semplice, può diventare complessa. Un modo per nascondere la complessità è l'**astrazione**. Un modo è dividere l'infrastruttura di rete in diversi **layer**.

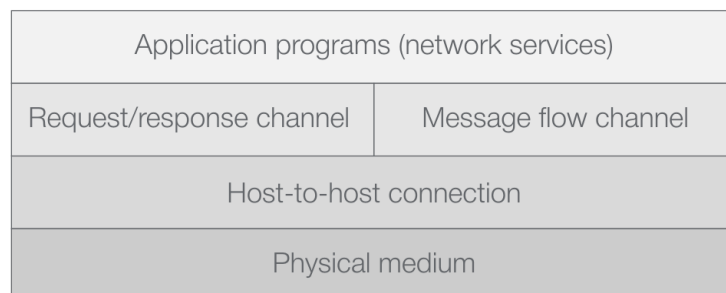


Figura 1.2: Esempio di layers.

1.3 Protocolli

Le comunicazioni richiedono collaborazione e cooperazione per arrivare ad un obiettivo comune, ossia dei **protocolli** da seguire.

Protocollo: insieme di regole e convenzioni seguite da entità, situate su nodi distinti, che intendono comunicare per svolgere un compito comune.

Ogni protocollo è formato da più parti, tutte utili alla comunicazione:

1. **Sintassi:** insieme e struttura di comandi, risposte e formato del messaggio.
2. **Semantica:** significato dei comandi, delle azioni e delle risposte quando trasmetti e ricevi messaggi.
3. **Timing:** specificazione delle sequenze temporali per comandi, messaggi e risposte.

Comunicazioni complesse richiedono, comunque, l'utilizzo di più protocolli che cooperano fra di loro. I protocolli sono i mattoni di ogni infrastruttura di rete, come l'esempio nella figura 1.2.

Ogni protocollo ha tre interfacce:

1. Due interfacce interne per comunicare con il livello superiore e con quello inferiore.
2. Una interfaccia esterna per comunicare con il protocollo di pari livello di un altro nodo.



Figura 1.3: Esempio di comunicazione fra protocolli.

1.3.1 Protocol Data Unit (PDU)

Ad ogni livello il messaggio consiste in due parti:

1. **Protocol Control Information (PCI):** ossia l'header.
2. **Service Data Unit (SDU):** ossia le informazioni.

L'insieme di queste due parti è il **PDU**.

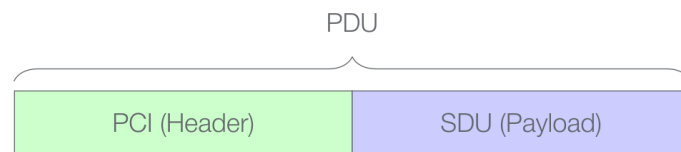


Figura 1.4: PDU.

Ad ogni layer viene incapsulato un header alle informazioni, partendo dal livello N fino al livello 1. Successivamente, sarà spaccettato e usato dall'altro host. La gerarchia dei protocolli è definita dall'architettura dell'infrastruttura di rete a layer.

Capitolo 2

Standard Suite

Per riuscire a comunicare fra nodi eterogenei in SW/HW sono necessari degli **standard**. I due principali sono:

- **ISO-OSI**.
- **TCP/IP**.

2.1 Modello ISO-OSI

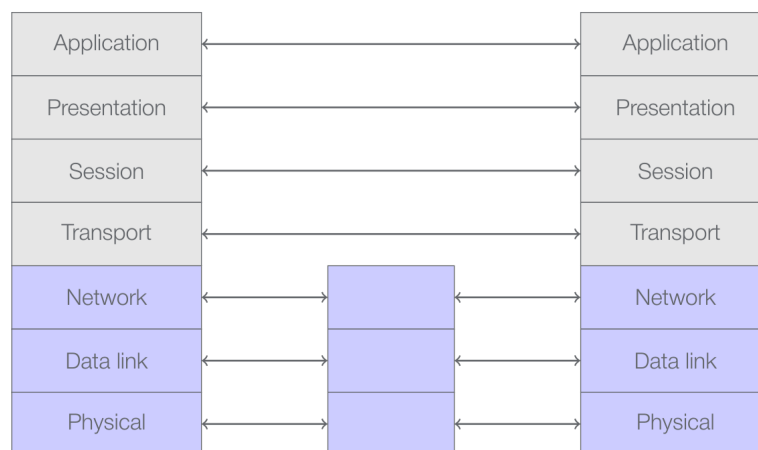


Figura 2.1: ISO-OSI Stack.

ISO (International Standard Organization): defined the specifications of what should have become the protocol standard for the interconnection of heterogeneous nodes.

OSI (Open Source Informaztion)

Lo stack **ISO-OSI** è formato da protocolli di **elaborazione** per gestire le applicazioni e da protocolli di **comunicazione** per gestire lo scambio di messaggi fra nodi di rete e per nascondere le caratteristiche del mezzo fisico di trasmissione. Ci sono in totale 7 livelli:

1. **Fisico**.
2. **Data-Link**.
3. **Rete**.
4. **Trasporto**.
5. **Sessione**.
6. **Presentazione**.
7. **Applicazione**.

2.1.1 Livello Fisico

Gestisce i dettagli meccanici ed elettrici della trasmissione fisica di un flusso di bit.

2.1.2 Livello Data-Link

Gestisce frame o pacchetti trasformando la semplice trasmissione in una linea di comunicazione priva di errori non rilevati.

2.1.3 Livello Rete

Fornisce connessioni e instradamento dei pacchetti nella rete.

2.1.4 Livello Trasporto

Esegue il controllo end-to-end della sessione di comunicazione, accesso alla rete da parte del client e trasferimento di messaggi tra client.

2.1.5 Livello Sessione

Consente agli utenti su macchine eterogenee di stabilire sessioni, implementando le funzioni di coordinamento, sincronizzazione e mantenimento dello stato.

2.1.6 Livello Presentazione

Risolve le differenze di formato che possono sorgere tra i diversi nodi della rete e gestisce la compressione dei dati, la sicurezza e l'autenticità dei messaggi tramite tecniche di crittografia.

2.1.7 Livello Applicazione

Fornisce un'interfaccia standard per i programmi applicativi che utilizzano la rete, mascherando le peculiarità e la complessità del sistema sottostante.

2.1.8 PDU dello stack ISO-OSI

Ogni livello dello stack ISO-OSI aggiunge un header al PDU, che sarà formato quindi dalla informazioni e 7 header diversi.

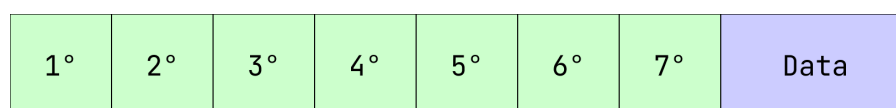


Figura 2.2: ISO-OSI PDU.

2.2 Modello TCP/IP

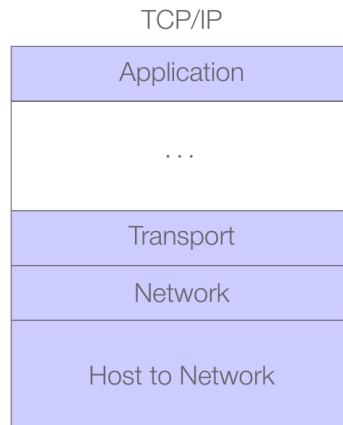


Figura 2.3: TCP/IP Stack.

TCP/IP è il modello più usato ed è di fatto lo standard, anche perchè implementato nei sistemi UNIX e ha molte API per creare applicazione network.

2.2.1 Livello Host-to-Network

Include i primi due livelli del modello ISO-OSI che però rimangono indipendenti. Qui ci sono protocolli come Ethernet, Token-Ring, 802.11, ecc...

2.2.2 Livello Rete

A questo livello viene usato il protocollo **IP**, che si occupa di mandare pacchetti da un nodo sorgente a un nodo destinatario. Tratta ogni pacchetto indipendentemente e non assicura che tutto sia stato inviato correttamente.

2.2.3 Livello Trasporto

Si occupa del multiplexing e demultiplexing dei messaggi fra ogni processo e aggiunge un rilevamento degli errori. I protocolli principali sono **TCP** e **UDP**.

2.2.4 Livello Applicazione

Il livello applicativo utilizza il livello di trasporto delle informazioni tra i processi in esecuzione sugli host terminali per creare applicazioni di rete. Alcuni protocolli usati sono **SSH**, **FTP**, **HTTP**, **SMTP**, ecc...

2.3 Circuit switching

Un circuito virtuale dedicato per ogni comunicazione, ha la necessità di riservare tutte le risorse end-to-end prima della trasmissione. Risorse dedicate e non è possibile la condivisione. Ci sono due metodi per dividere le risorse:

- **Frequency Based Methods (FDM).**
- **Time-based methods (TDM).**

2.4 Packet Switching

I dati vengono suddivisi in parti e inviati attraverso la rete. Ogni pacchetto utilizza tutta la capacità di trasmissione di un collegamento. Le risorse vengono utilizzate in base alle necessità e non in base alla prenotazione.

Capitolo 3

Livello Host-to-Network

3.1 Definizione e tecnologie

Il livello **H2N** risolve due problemi, quello delle interconnessioni fra due host e quello dello scambio di dati fra due host. Nello specifico il livello **Fisico** si occupa di connettere gli host con diversi mezzi di trasmissione, invece il livello **Data-Link** si occupa del framing, del controllo e della correzione di errori e di inviare i **frame** solo fra nodi della **LAN**. Ci sono più tipi di trasmissione:

1. **Broadcast**: comunicazione tra un singolo mittente e tutti gli altri nodi.
2. **Multicast**: comunicazione tra un singolo mittente e un gruppo di destinatari.
3. **Anycast**: comunicazione tra un singolo mittente e almeno un destinatario in un gruppo.
4. **Unicast**: comunicazione tra un singolo mittente e un singolo destinatario.

Il protocollo H2N è implementato nella **NIC (Network Interface Card)**, tutti i dispositivi della rete per comunicare devono averne una.

LAN: area fisicamente limitata a cui sono connessi uno o più dispositivi. Misurata in $\frac{\text{Bit}}{\text{s}}$. Tutte le tecnologie seguono lo standard **IEEE Standard 802**, come **Ethernet (IEEE 802.3)** e **WLAN (IEEE 802.11x)**.

3.2 Ethernet

Creata a metà degli anni '70 da Bob Metcalfe, che propose Ethernet nella sua tesi di dottorato.

Ethernet: metodo di trasmissione usato globalmente perché economico ed efficiente, funziona bene con i protocolli dello stack **TCP/IP**. Connessione di tipo **broadcast**, quindi tutti sono collegati allo stesso canale di comunicazione e anche trasmissione di tipo **broadcast**, quindi quando viene inviato un messaggio nella **LAN**, tutti i nodi ad essa collegati lo ricevono.

Gli host utilizzano un indirizzo hardware o indirizzo **MAC (Media Access Control)** per capire chi è il destinatario dei dati.

MAC: indirizzo unico della **NIC** a **48 bit** che viene inserito nella ROM quando viene realizzata. La sua struttura è formata da:

- 3 bytes (24 bit) → **Organizationally Unique Identifier (OUI)**, di solito il produttore della NIC.
- 3 bytes (24 bit) → **NIC ID**, all'interno dell'OUI.

Ogni dispositivo per fare parte della LAN deve avere una NIC e quindi avrà anche un indirizzo MAC unico.

L'indirizzo broadcast è FF-FF-FF-FF-FF-FF.

Quindi:

- Quando un host desidera trasmettere un pacchetto, inserisce l'indirizzo MAC del destinatario e lo trasmette nella LAN.
- Se l'indirizzo di destinazione del pacchetto corrisponde all'indirizzo MAC dell'host ricevente, quest'ultimo accetta il pacchetto e lo inoltra al protocollo successivo.
- Se l'indirizzo di destinazione non corrisponde all'indirizzo MAC dell'host ricevente, quest'ultimo scarta il pacchetto.

ESEMPIO:

```
donati@linux > ifconfig eth0
eth0 Link encap:Ethernet HWaddr 00:10:5A:9D:88:AF
inet addr:192.168.1.3 Bcast:192.168.1.255 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:92 errors:0 dropped:0 overruns:0 frame:0
TX packets:52 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:14162 (13.8 KiB) TX bytes:6744 (6.5 KiB)
Interrupt:9 Base address:0xfc00
```

00:10:5A:9D:88:AF è l'indirizzo MAC, invece, 192.168.1.3 è l'indirizzo IP.

3.2.1 Frame Ethernet

I pacchetti scambiati al livello host to network sono chiamati **frame**. Per tecnologie e topologie diverse, il formato di ogni frame è sempre lo stesso.



Figura 3.1: Frame Ethernet.

3.2.2 Preamble

Campo da 8 byte, di cui i primi 7 byte valgono tutti 10101010 e l'ultimo byte vale 10101011. I primi 7 byte servono per attivare e sincronizzare i ricevitori, due 1 consecutivi nell'ultimo byte indicano che la sincronizzazione è finita e il contenuto del frame sta per arrivare.

3.2.3 Indirizzo di Destinazione e di Origine

Due campi da 6 byte ciascuno, che indicano l'indirizzo MAC di destinazione e l'indirizzo MAC di origine. Quando un ricevitore riceve un MAC equivalente al suo passa il contenuto al layer di Rete, altrimenti lo scarta.

3.2.4 Tipo

Campo da 2 byte che indica il tipo di protocollo che verrà usato p al layer di Rete (IP, Nevel IPX, Apple Talk, ecc), è usato anche per supportare i protocolli **ARP** e **RARP**.

3.2.5 Data e Dimensione

Contiene i dati (**IP Datagram**), l'unità massima di trasmissione per Ethernet (**MTU**) è 1500 byte, altrimenti va frammentato, invece, quella minima è 46 byte, se minore di questa cifra va aggiuntoun padding.

3.2.6 CRC (Cyclic Redundancy Check)

Campo da 4 byte che consente all'adattatore che riceve i dati di rilevare la presenza di un errore nei bit del frame ricevuto.

3.3 Protocollo ARP

L'indirizzo IP non è riconosciuto dall'hardware, quindi il protocollo ARP si occupa di trasformare l'IP di un host nella stessa sua LAN nel suo corrispettivo indirizzo MAC. Nello specifico usa due tipi di messaggi:

1. Messaggio broadcast di richiesta con l'indirizzo IP.
2. Messaggio unicast di risposta con il corrispettivo indirizzo MAC.

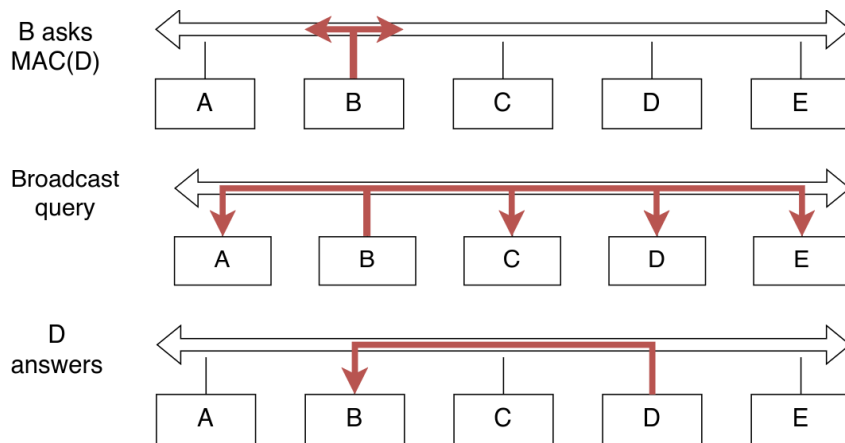


Figura 3.2: Protocollo ARP.

3.3.1 ARP Cache

Per ridurre il traffico di rete causato dallo scambio di messaggi ARP, ogni host memorizza temporaneamente nella cache le risoluzioni IP/MAC nella propria tabella.

IP	MAC	TTL
222.222.222.221	88:B2:2F:54:1A:0F	13:45:00
222.222.222.223	5C:66:AB:90:75:B1	13:52:00

ESEMPIO:

```
donati@linux > sudo arp
Address HWtype Hwaddress Flags Mask Iface
ns1.ing.unimo.it ether 00:01:03:11:6B:63 C eth0
info-gw.ing.unimo.it ether 00:0A:57:05:A0:00 C eth0

donati@linux > ping brandy.ing.unimo.it
[...]
```

```
donati@linux > sudo arp
Address HWtype Hwaddress Flags Mask Iface
brandy.ing.unimo.it ether 00:11:11:5A:EB:41 C eth0
ns1.ing.unimo.it ether 00:01:03:11:6B:63 C eth0
info-gw.ing.unimo.it ether 00:0A:57:05:A0:00 C eth0
```

3.3.2 RARP

Per ottenere l'indirizzo IP a partire da un indirizzo MAC si può usare il protocollo RARP, che funziona con la stessa struttura e nello stesso modo del protocollo ARP.

3.4 Interconnessione fra LAN

Nel fare delle reti LAN molti grandi, possono causarsi molti problemi, dalle collisioni a il limite di lunghezza dello standard 802.3.

Per questo ci sono diversi tipi di dispositivi di rete, che dipendono da topologia, numero di host connessi, ecc:

- **Hub.**
- **Bridge.**
- **Switch.**
- **L3 Switch.**

3.4.1 Hub

Dispositivo di livello fisico con due o più interfacce. Ripete i bit ricevuti su un'interfaccia a tutte le altre interfacce, ogni nodo connesso costituisce un segmento LAN.

3.4.2 Bridge

Dispositivo che opera a livello 2 nell'Ethernet frame, analizza l'header e distribuisce selettivamente in base al MAC di destinazione. Per trasmettere usa il protocollo CSMA/CD.

3.4.3 Hub VS Bridge

Il Bridge è più intelligente di un Hub, dato che è in grado di analizzare gli header e leggere il MAC, quindi, usando delle funzioni da filtro. Inoltre, è utile perchè elimina le collisioni di dominio, aumenta il throughput e non necessita di cambiamenti sugli host. I bridge apprendono quali host sono raggiungibili tramite quali interfacce, infatti:

1. Mantengono tabelle di filtro create automaticamente senza la necessità dell'intervento degli amministratori di rete.
2. Quando viene ricevuto un frame, il bridge apprende la posizione del mittente.
3. Registra la posizione del mittente nella tabella di filtro usando:
 - Indirizzo MAC dell'host.
 - Interfaccia del bridge.
 - Time-To-Live (TTL).

I record obsoleti nella tabella di filtro vengono scartati in base alla scadenza del TTL.

ESEMPIO:

```
if frame.destination in filter_table then
    dest_lan_segment = lookup_filter_table(frame.destination)
    forward_to_lan_segment(frame, dest_lan_segment)
else
    {send frame to all ports except incoming port}
    flood(frame)
endif
```

3.4.4 Switch

Si tratta di bridge ad alte prestazioni con numerose interfacce. Ci sono più interfacce eterogenee (10/100/1000 Mbps) condivise e dedicate, senza collisioni con accesso dedicato e full duplex.

3.4.5 Bridge VS Switch

Concettualmente sono lo stesso dispositivo, il bridge è software, lo switch è hardware.

3.4.6 Bridge in Linux

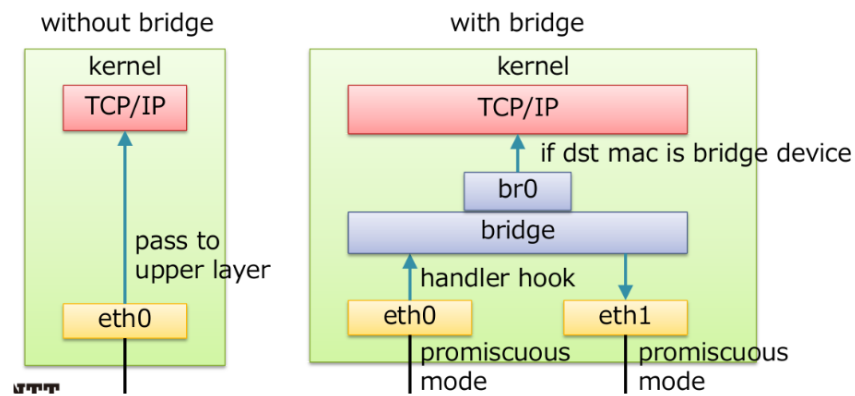


Figura 3.3: Bridge in Linux.

I bridge possono essere configurati tramite il file `/etc/network/interfaces` oppure:

Creazione Bridge:

```
brctl addbr <bridge>
ip link add <bridge> type bridge ...
```

Aggiungere interfaccie al Bridge:

```
brctl addif <bridge> <iface>
ip link set <iface> master <bridge>
```

Vedere i Bridge configurati:

```
brctl show <bridge>
```

Vedere la lista dei MAC conosciuti dal Bridge:

```
brctl showmacs <bridge>
```

3.5 Virtual LAN

Lo standard 802.1Q definisce le specifiche per definire più reti locali virtuali (VLAN) distinte, utilizzando la stessa infrastruttura fisica. Ogni VLAN agisce come se fosse separata dalle altre:

- I pacchetti broadcast sono confinati all'interno della VLAN.
- Le comunicazioni a livello 2 sono confinate all'interno della VLAN.
- Le VLAN non possono comunicare fra di loro, se non a livello 3 tramite routing.

Le VLAN hanno diversi vantaggi tra cui:

1. **Risparmio:** evitano di dover creare nuove infrastrutture fisiche.
2. **Performance:** evita che i frame vengano inviati a destinazioni che non devono riceverli.
3. **Security:** un utente connesso a una VLAN non ha modo di vedere le altre.
4. **Flessibilità:** il movimento fisico di un utente non richiede dei cambiamenti fisici ma logici.

Per poter funzionare e quindi riconoscere le VLAN bridge e switch devono essere conformi allo standard 802.1Q. Devono essere in grado di fare queste tre azioni:

1. **Ingresso:** il bridge deve saper riconoscere a quale VLAN appartiene il frame che arriva da una porta.
2. **Trasmissione:** il bridge deve conoscere quale porta il frame deve essere trasmesso, in base alla VLAN alla quale appartiene.
3. **Uscita:** il bridge deve essere in grado di far uscire il frame in modo che la sua VLAN venga riconosciuta da altri bridge.

Le Untagged VLAN (private), dove a ogni porta appartiene una VLAN, non richiedono la conformità allo standard 802.1Q, ma solo che lo switch supporti la sua configurabilità.

3.5.1 Tagged VLAN

Lo standard 802.1Q permette di condividere lo stesso accesso fisico e che il bridge sia in grado di riconoscere le varie VLAN. Per questo, sono stati aggiunti 4 byte al frame Ethernet, per specificare i dati della VLAN:

- **TPI (Tag Protocol Identifier):** due byte, di valore 81 00, che identificano il protocollo 802.1Q.
- **TCI (Tag Control Information):** due byte che specificano le informazioni sul tag:
 - I primi tre (**user priority**) indicano la priorità del frame.
 - Il quarto bit (**CFI**) se è 1 indica che il frame viene da una token ring LAN.
 - I restanti dodici bit (**VID** indica il tag (da 0 a 4095). Il tag 0 e il tag 4095 sono riservati e non possono essere usati.

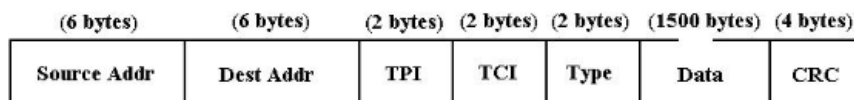


Figura 3.4: Frame standard 802.1Q.



Figura 3.5: Campo TCI.

Capitolo 4

Livello IP

4.1 Importanza e compiti del livello Network

Viene definita l'unità di trasferimento dati, ossia il Datagramm (da 64 a 1500 byte). Inoltre, garantisce l'indirizzamento univoco degli host, infatti, tutti gli host connessi a Internet devono essere identificati tramite un Indirizzo IP anche grazie al fatto che viene chiarita l'architettura di Internet. Tutti i router e i sistemi autonomi che compongono Internet possono essere raggiunti tramite funzioni di routing, ossia algoritmi determinano il percorso dei dati all'interno di una rete geografica. La consegna dei datagrammi però non è affidabile (best-effort).

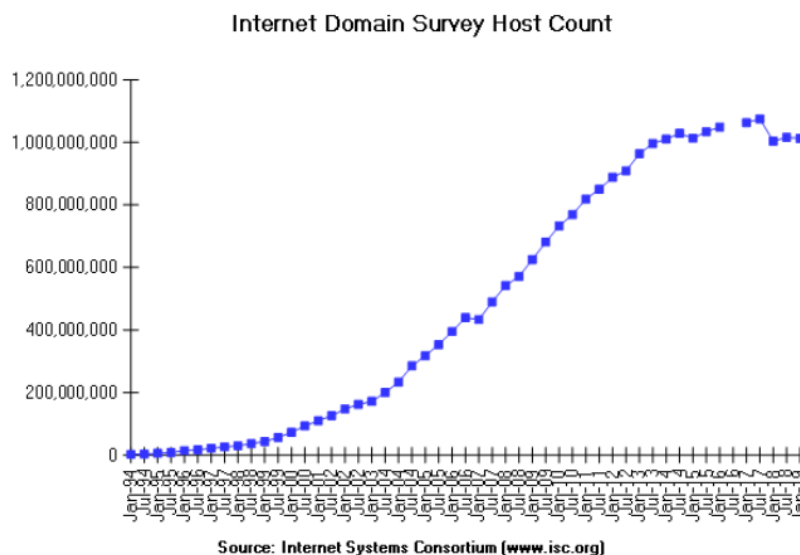


Figura 4.1: Grafico degli IPv4 univoci su Internet.

Non tutti i protocolli a livello Network forniscono un sistema di consegna non affidabile, per esempio la telefonia fornisce un approccio **connection-oriented**, rispetto all'approccio **connection-less** di Internet. Infatti:

- **Circuit Switching:** è necessario riservare tutte le risorse end-to-end prima della trasmissione:
 - Non è possibile condividere le risorse assegnate.
 - Per ogni chiamata è necessaria una fase di configurazione.
 - Prestazioni garantite in base al tipo di risorse riservate.
- **Packet Switching:** nessuna risorsa riservata:
 - Ogni utente può utilizzare tutte le risorse disponibili.
 - Nessuna configurazione richiesta.
 - Rischio di conflitti/congestione.

4.2 Protocollo IP

Protocollo IP: è un protocollo di comunicazione a livello network, ideato da Vint Cerf e Bob Kahn nel maggio del 1974 e pubblicato sull'IEEE. La prima versione pubblicata è stato IPv4, il suo successore è IPv6.

Il **protocollo IP** fornisce alcuni servizi molto importanti:

- **Indirizzamento host univoco:** fornisce un grafico praticamente completo di tutti gli host (indirizzi IP).
- **Unità di trasferimento dati:** definisce l'unità di informazione di base utilizzata da Internet per trasferire dati.
- **Funzioni di Routing:** sceglie il percorso nella rete nel quale inviare i pacchetti.

La consegna dei pacchetti, però, non è affidabile:

- **Consegna connectionless:** ogni pacchetto è trattato in modo indipendente rispetto agli altri.
- **Consegna Best-Effort:** tentativo di consegnare ogni pacchetto.
- **Consegna non garantita:** i pacchetti possono essere persi, duplicati, ritardati o consegnati fuori ordine.

4.2.1 Datagram

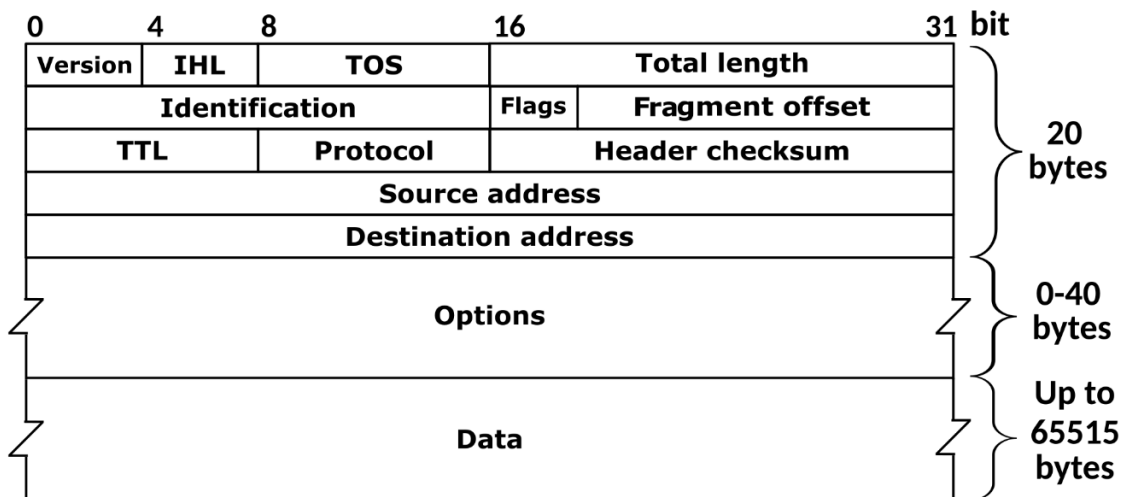


Figura 4.2: Composizione di un Datagram IP.

Il **Datagram IP** è formato da due parti principali, l'header e i dati che effettivamente vengono inviati. L'header IP a sua volta è formato da diversi campi:

- **Version:** campo da 4 bit con la versione del protocollo IP usato per la creazione del Datagram.
- **IP Header Length (IHL):** campo da 4 bit che indica la lunghezza dell'header (massimo 20 byte).
- **Total Length:** campo da 16 bit con la lunghezza del Datagram IP (massimo 64kb).
- **Type of Service (TOS):** campo da 8 bit che specifica come il Datagram deve essere trattato, i bit di questo campo sono divisi così:
 - **Precendence:** 3 bit che specificano l'importanza del datagram.
 - **Delay (D):** 1 bit.
 - **Throughput (T):** 1 bit.
 - **Affidabilità: (R):** 1 bit.

- **Identificazione:** campo da 16 bit che identifica il Datagram.
- **Flags:** controllo della frammentazione:
 - **Reserved.**
 - **Don't Fragment (DF).**
 - **More Fragment (MF).**
- **Fragment Offset:** indica la posizione del frammento nel datagram originale.
- **Time To Live (TTL):** campo da 8 bit che contiene un contatore (NB: non è un tempo) che indica quanto un datagram può circolare su Internet. Viene decrementato ogni volta che incontra un router e appena arriva a 0 viene eliminato.
- **Protocol:** campo da 8 bit che indica che tipo di protocollo di più alto livello può usare il datagram.
- **Header Checksum:** usato per controllare l'integrità dei dati dell'header del datagram.
- **Indizzo sorgente:** campo da 32 bit con l'indirizzo IP del mandante.
- **Indirizzo di destinazione:** campo da 32 bit con l'indirizzo IP del destinatario.
- **Opzioni:** campo da 32 bit usato per testing e debugging.
- **Padding:** necessario se le opzioni non sono tutte a 32 bit.

4.3 Ip Addressing

Ogni indirizzo IPv4 ha 32 bit, diviso in 4 campi da 8 bit ciascuno e separati da un punto e sono formati dalla coppia NetID, HostID. Indirizzi nella stessa rete avranno lo stesso NetID ma diverso HostID.

4.3.1 Classe di Indirizzi IP

Per capire quanti bit deve avere il NetID e quanti bit deve avere l'HostID si deve guardare a che classe appartengono:

- **Classe A:** un byte dedicato alla NetID e tre byte dedicati all'HostID. Da 0.0.0.0 a 127.255.255.255.
- **Classe B:** due byte per la NetID e due byte per l'HostID. Da 128.0.0.0 a 191.255.255.255.
- **Classe C:** tre byte per la NetID e un byte per l'HostID. Da 192.0.0.0 a 223.255.255.255.
- **Classe D:** indirizzi multicast riservati. Da 224.0.0.0 a 239.255.255.255.
- **Classe E:** indirizzi riservati per usi futuri. Da 240.0.0.0 a 255.255.255.254.

Ogni IP può essere assegnato dinamicamente o staticamente. Ogni dispositivo può avere più interfacce di rete e quindi avere più IP. Inoltre, ci sono alcuni indirizzi speciali:

- **Indirizzo della rete:** gli indirizzi con HostID tutto a 0 indicano l'indirizzo della rete.
- **Indirizzo Broadcast:** indirizzi con HostID tutto a 1 funzionano come indirizzo di Broadcast per tutta la rete.
- **Questo Host su questa rete:** l'indirizzo 0.0.0.0 è usato per fare il boot dell'host.
- **Indirizzo di Loopback:** l'indirizzo di classe A con NetID 127 è usato per fare test in locale. Per esempio l'indirizzo Localhost è 127.0.0.1.

4.4 Subnetting e Supernetting

La divisione degli indirizzi è molto rigida, per rendere tutto più semplice e maneggevole si possono usare due metodi:

- **Sottoclassi:** per le organizzazioni.
- **Soureclassi:** per gli ISP.

Quindi è necessario definire in modo flessibile e logico la coppia **NetID**, **HostID**. Infatti, per definire i bit della NetID viene usata una network mask da 4 byte. Se si fa una operazione di AND fra l'ip e la mask, gli 1 indicheranno la parte dedicata alla NetID e gli 0 indicheranno la parte dedicata all'HostID.

4.4.1 Subnetting

Un'organizzazione può suddividere il proprio spazio di indirizzi host in gruppi chiamati sottoreti:



Figura 4.3: Esempio di indirizzo di una subnet.

ESEMPIO:

Dati:

- **IP address:** 192.168.81.56
- **Netmask:** 255.255.255.240

Per calcolare il NetID:

IP Address **10011100.10011010.01010001.00111000** = **156.154.81.56**

Netmask **11111111.11111111.11111111.11110000** = **255.255.255.240**

Result **10011100.10011010.01010001.00110000** = **156.154.81.48**

4.5 Architettura di Internet

Internet non è un insieme di router sparsi a caso per il mondo che sono interconnessi tra loro.

Internet: la definizione di **Internet** dipende dal punto di vista:

- Dal punto di vista delle **applicazioni di rete**, ossia un'entità trasparente, nella maggior parte dei casi.
- Dal punto di vista **fisico**, un insieme di componenti interni, in cui ogni nodo è caratterizzato da un indirizzo IP.
- Dal punto di vista **organizzativo**, un insieme di sistemi autonomi (considerando i **router**) e un insieme di nomi e domini (considerando gli **host**).

Ha alcuni principi:

- **Sopravvivenza:** se esiste un percorso tra i due host, la comunicazione deve essere in grado di continuare.
- **Forma a clessidra:** IP fa ipotesi minime sui mezzi di trasferimento dati sottostanti. Deve funzionare per tutti i tipi di applicazioni di rete e con un approccio senza stato.
- **Sistemi autonomi:** ogni rete è di proprietà e gestita da un'entità diversa.

4.5.1 Autonomous System

AS: un insieme di **reti IP** e **router** sotto il controllo di un'organizzazione all'interno della quale viene utilizzata una politica di **routing** interna. Tutti i router all'interno dello stesso **AS** utilizzano lo stesso algoritmo di instradamento dei messaggi e si scambiano continuamente informazioni con gli altri router. Dall'esterno, i **sistemi autonomi** sono visti come un'unica entità.

Gli AS sono interconnessi fra di loro in due modi:

1. Attraverso punti di peering privati.
2. Attraverso punti di scambio Internet (IXP).

4.6 Routing

Inoltrare pacchetti all'interno di una rete è un problema complesso, per questo si può dividere in sottoproblemi:

1. Ad ogni hop, inoltra il pacchetto a un altro nodo in modo che si avvicini alla destinazione (IP forwarding).
2. Mantenere le informazioni aggiornate per poter risolvere il primo sottoproblema (Gestione delle tabelle di routing).

IP Forwarding: operazione eseguita da ogni **router**. Il router successivo deve appartenere alla **rete** a cui il router è direttamente connesso. Per inoltrare i pacchetti, l'indirizzo di destinazione viene estratto dall'header del **pacchetto** e viene utilizzato come indice nella tabella di routing.

L'IP forwarding ha alcune caratteristiche importanti:

- **Indipendenza dal mittente:** il routing del next-hop non dipende dal mittente del pacchetto o dal percorso che il pacchetto ha percorso fino a quel punto.
- **Routing universale:** la tabella di routing deve contenere un router next-hop per ogni destinazione.
- **Routing ottimale:** il router next-hop viene scelto in modo da minimizzare il percorso verso la destinazione con l'utilizzo di algoritmi di routing.

Tabelle di Routing: ogni host e ogni router possiede una tabella di routing. Ogni riga indica il next-hop per ogni possibile destinazione.

Esistono due tipi di **Routing**:

1. Statico:

- La tabella di routing non viene modificata dal router.
- L'amministratore di rete deve inserire o modificare le righe della tabella di routing.
- Impossibilità di reagire automaticamente ai cambiamenti topologici.
- Da utilizzare in caso di reti di piccole dimensioni con poche modifiche.

2. Dinamico:

- La tabella di routing viene modificata dal router al variare delle condizioni di rete.
- Lo scambio di informazioni e il calcolo dei valori della tabella di routing avvengono tramite protocolli di routing.

ESEMPIO DI IP FORWARDING:

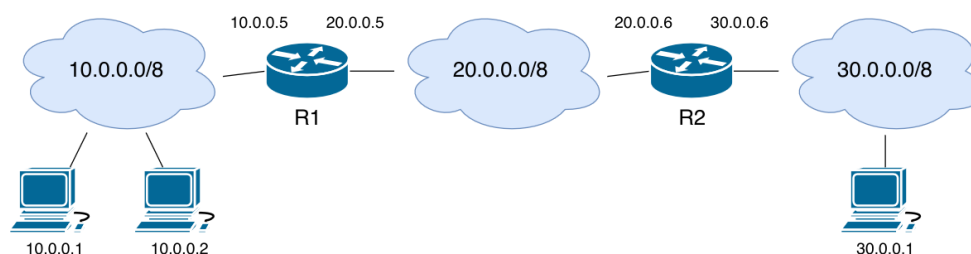


Figura 4.4: Esempio di indirizzamento di una subnet.

Si hanno queste tabelle di routing per quanto riguarda i router:

DESTINATION	NEXT HOP	DESTINATION	NEXT HOP
10.0.0.0/8	direct	10.0.0.0/8	20.0.0.5
20.0.0.0/8	direct	20.0.0.0/8	direct
30.0.0.0/8	20.0.0.6	30.0.0.0/8	direct

E queste tabelle di routing per quanto riguarda gli host:

DESTINATION	NEXT HOP	DESTINATION	NEXT HOP
10.0.0.0/8	direct	10.0.0.0/8	direct
default	10.0.0.5	default	30.0.0.6

Se volessi mandare un pacchetto dall'host **10.0.0.1** all'host **10.0.0.2** sarebbe necessario solamente il protocollo ARP, invece, se volessi mandare un pacchetto dallo stesso host sorgente all'host **30.0.0.1**, dovrei passare prima per il router R1 (sempre tramite il protocollo ARP) e successivamente andare al router R2 per raggiungere l'host di destinazione.

4.6.1 Teoria del Routing

L'obiettivo del routing è stabilire il percorso ottimale, ossia quello con costo minore che colleghi i due host che vogliono comunicare. Per formulare un algoritmo di routing, la rete viene modellata utilizzando un grafo pesato $G(N, E)$ dove:

- I nodi **N** rappresentano i router (o gli AS).
- Gli archi **E** rappresentano le connessioni tra i router.
- Le etichette degli archi **E** rappresentano il "costo" delle connessioni tra i router.

Si dividono in due categorie principali:

1. **Algoritmi di routing distribuiti:** nessun nodo possiede informazioni complete sul costo di tutti i collegamenti nella rete.
2. **Algoritmi di routing centralizzati:** ogni nodo possiede informazioni globali sulla rete.

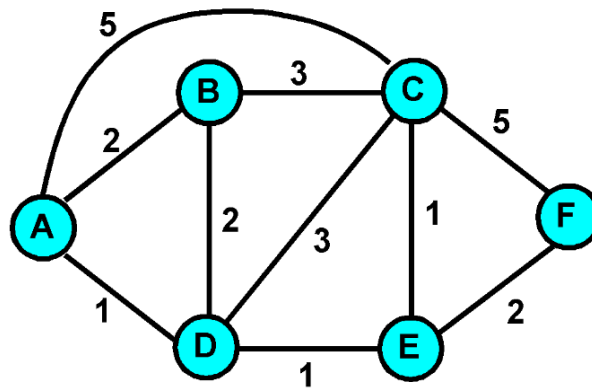


Figura 4.5: Grafo pesato.

Ci sono alcuni fattori che possono influenzare il routing, come la topologia, traffico, fallimenti e policies.

4.6.2 Distan Vector Protocol

Calcolo **distribuito** del **next hop**, è un algoritmo adattivo rispetto ai cambiamenti di stato. Esempio di implementazione principale è l'algoritmo distribuito di Bellman-Ford.

Algoritmo di Bellman-Ford

Compiti di ciascun nodo:

- Aggiornamento del vettore di distanza in risposta alle variazioni di costo sui collegamenti adiacenti.
- Invio di un aggiornamento ai nodi adiacenti se il loro vettore di distanza cambia.

Dati su ciascun nodo $\mathbf{x}$:

- $c(x, v)$ costo del collegamento tra il nodo $\mathbf{x}$ e il nodo vicino $\mathbf{v} \in \mathbf{V}$.
- $\mathbf{Dx} = \{Dx(y) : y \in \mathbf{N}\}$ vettore delle distanze dal nodo $\mathbf{x}$ a tutti i nodi $\mathbf{y} \in \text{rete } \mathbf{N}$.
- $\mathbf{Dv} = \{Dv(y) : y \in \mathbf{N}\}$ vettori di distanza dei nodi vicini $\mathbf{v} \in \mathbf{V}$ di $\mathbf{x}$.

Bellman-Ford serve per calcolare il costo minimo da $\mathbf{x}$ a $\mathbf{y}$:

$$Dx(y) = \min_v \{c(x, v) + Dv(y)\}$$

Dove $\min_v$ è calcolato su tutti i nodi $\mathbf{v}$ vicini al nodo $\mathbf{x}$. Tra tutti i nodi $\mathbf{v}$ adiacenti al nodo $\mathbf{x}$, il percorso da scegliere è quello che porta da $\mathbf{v}$ a $\mathbf{y}$ con il costo più basso.

INIZIALIZZAZIONE:

```
for all y in N do
  if y in V then
    Dx(y) = c(x, y)
  else
    Dx(y) = infinito

  Send vector Dx to all v in V
```

LOOP:

```
hile True do
  # Wait for change in neighbor link cost or update
  for all y in N do
    Dx(y) = minv{c(x, v) + Dv(y)}

    if exist y in N : Dx(y) changed then
      Send vector Dx to all v in V
```

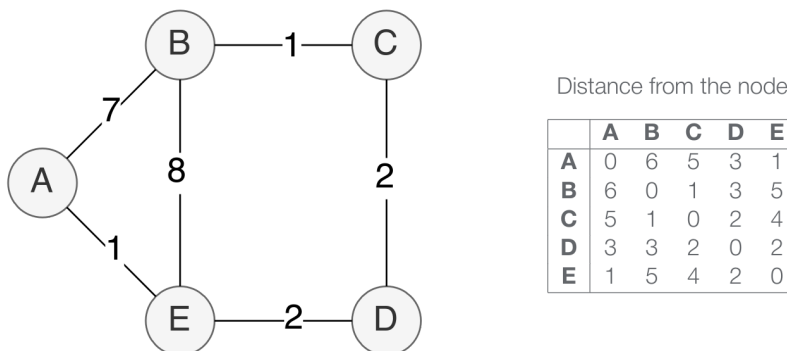


Figura 4.6: Esempio di tabella di Routing con Bellman-Ford.

4.6.3 Problemi

Se il costo di un percorso aumenta e il percorso alternativo minimo contiene il percorso che è aumentato, il pacchetto non arriverà ma a destinazione ma farà avanti e indietro. Questo è chiamato **Rebound Effect**. Se un nodo è connesso tramite un collegamento e il collegamento si guasta, allora non si avrà stabilizzazione e si avrà un problema **Count-to-Infinity**.

Le possibili soluzioni sono tre:

1. Scegliere un **threshold** per rappresentare il valore "infinito".
2. **Split Horizon**: è necessario differenziare i vettori di distanza inviati ai nodi adiacenti. Il vettore di B inviato a C non conterrà le destinazioni raggiungibili tramite C.
3. **Split Horizon with poisoned reverse**: se B raggiunge A passando per C, B avvertirà C che la sua distanza da A è infinita.

4.6.4 Algoritmi Link State

Sono algoritmi **centralizzati**, necessitano di conoscere la **topologia** e il **costo** di ogni collegamento. Tutti i nodi hanno una visione identica e completa della rete e fanno diverse cose:

1. Calcolano lo stato dei loro collegamenti.
2. Trasmettono periodicamente l'identità e i costi dei loro collegamenti connessi.
3. Calcolano i percorsi a costo minimo verso tutti gli altri nodi della rete utilizzando l'algoritmo di Dijkstra.

Tutte queste informazioni sono contenute nel **Link State Packet**, nello specifico:

- Node ID.
- Lista dei vicini e costi dei collegamenti.
- Time To Live (TTL).
- Sequence Number per riconoscere eventuali errori.

PROPAGAZIONE DEI PACCHETTI:

```
if P is most recent packet form j then
    save P to database
    for all links != incoming link of P do
        Forward P
    else
        Discard P
```

Algoritmo di Dijkstra

Algoritmo iterativo, quindi, all'iterazione k , il nodo i conosce il percorso a costo minimo verso k nodi di destinazione. Abbiamo:

- $c(i, j)$ costo del collegamento tra il nodo i e il nodo j .
- $D(v)$ costo minimo del percorso verso il nodo v (minimo per l'iterazione corrente).
- $p(v)$ predecessore immediato di v lungo il percorso a costo minimo verso v .
- N gruppo di nodi il cui percorso a costo minimo è definitivamente noto.

INIZIALIZZAZIONE:

```
N = {u}
for all nodes v do
    if v is adjacent to u then
        D(v) = c(u, v)
    else
        D(v) = infinito
```

LOOP:

```

repeat
  #Compute for all adjacent nodes i not in N the cost D(i)
  Add to N the node w with the minimum cost D(w)
  for all Nodes v adjacent to w and not in N do
    # new cost to v is the old one or the lowest cost to w plus the cost from
    w to v
     $D(v) = \min\{D(v), D(w) + c(w, v)\}$ 
until all the nodes of the graph in N

```

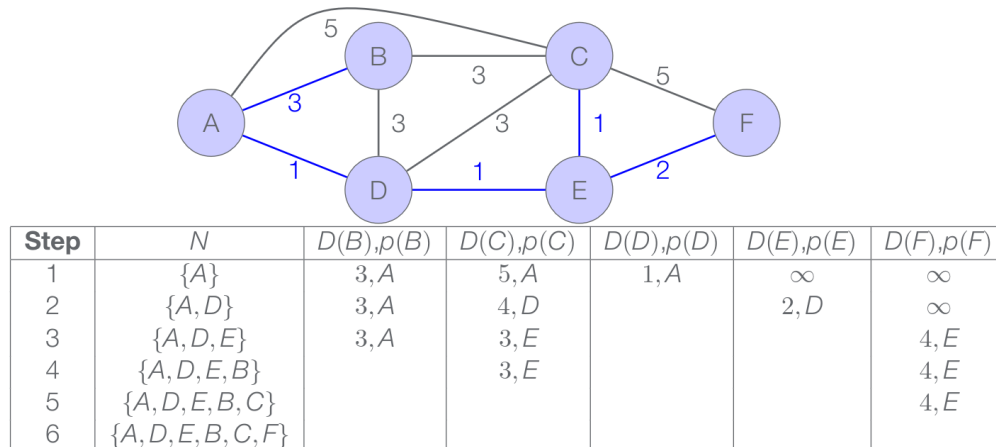


Figura 4.7: Esempio di tabella di Routing con Dijkstra.

4.6.5 Differenze fra Distan Vector Protocol e Algoritmi Link State

Non ci sono algoritmi migliori di altri, gli algoritmi **Link State** sono usati spesso all'interno degli **AS**, invece, i **Distant Vector Algorithm** sono più usati fra **AS**.

Distan Vector Protocol:

- Tutto ciò che è noto viene propagato solo ai vicini.
- Messaggi potenzialmente molto larghi.
- Numero di messaggi piccolo.
- Lenta convergenza e rischio di Rebound Effect o delay elevati.
- Requisiti di memoria piccoli.
- Calcoli del router basati su calcoli di altri router.

Algoritmi Link State:

- Le informazioni sono passate a tutti.
- Messaggi piccoli.
- Numero elevato di messaggi ($O(n)$ dove n è il numero di nodi del grafo).
- Convergenza molto veloce.
- Requisiti di memoria elevati.
- Ogni router calcoli la route del percorso indipendentemente.

4.7 Routing a scala geografica

Ogni **AS** ha un numero di identificazione, compreso tra 1 e 64511, assegnato da un'autorità di registrazione **Internet**. I numeri tra 64512 e 65535 sono riservati.

Per il routing dentro gli AS (**Intra-AS routing**) si usa un **Interior Gateway Protocol**, cioè i router di un AS possono avere informazioni complete su tutti gli altri router dell'AS. Per il routing fra AS (**Inter-AS routing**) si usa un **External Gateway Protocol**.

Esistono diversi tipi di AS:

- **Stub:** AS con una sola connessione a un altro AS.
- **Multi-Homed:** un AS è connesso con diversi altri AS, ma accetta solo traffico locale.
- **Transit:** un AS connesso a diversi AS che possono fungere da intermediari per l'AS a cui è connesso.

I protocolli di routing principali sono:

- **Border Gateway Protocol (BGP)** per il routing **Inter-AS**.
- **Routing Information Protocol (RIP)** per il routing **Intra-AS**, come **Distance Vector Protocol**, non più usato.
- **Open Shortest Path First (OSPF)** per il routing **Intra-AS**, come **Link State Protocol**, quello che viene usato ora.

4.7.1 Border Gateway Protocol (BGP)

Protocollo delle backbones di Internet usato da ISP per gestire il traffico tra i sistemi autonomi in modo completamente decentralizzato. Ha alcune funzioni principali:

- Scambio di informazioni di raggiungibilità tra AS vicini, chiamati **peer**.
- Propaga le informazioni di raggiungibilità a tutti i router all'interno di un AS con un meccanismo distribuito basato sull'algoritmo **Path Vector**.
- Determina i percorsi migliori in base alle informazioni di raggiungibilità e alle **politiche di instradamento**.

Per comunicare vengono utilizzate connessioni TCP semi-permanenti per consentire ai router vicini di comunicare, creando di fatto una **sessione BGP**. Le sessioni possono essere di due tipi:

1. **External session (E-BGP)** fra router (**Border Router**) di diversi AS.
2. **Internal session (I-BGP)** fra router (Transit Router) dello stesso AS.

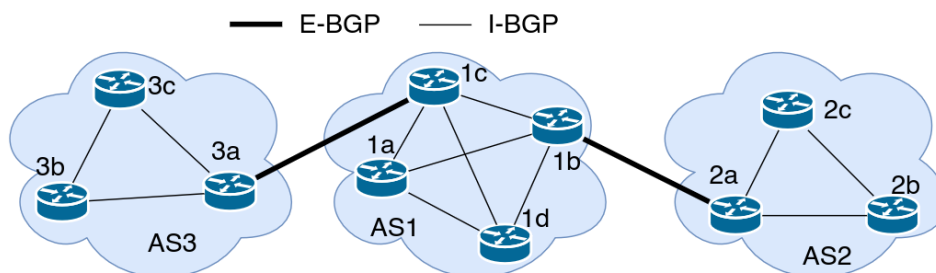


Figura 4.8: Routing BGP.

Algoritmo Path Vector

BGP utilizza Path Vector, una variante dell'algoritmo distribuito Distance Vector Protocol. Ogni aggiornamento di routing contiene non solo informazioni adiacenti, ma anche informazioni sull'intero percorso verso la destinazione tramite gli AS. Utilizza metriche e politiche locali nei processi decisionali e garantisce l'assenza di cicli.

4.7.2 Open Short Path First (OSPF)

Il routing si basa sull'algoritmo centralizzato **Link State**. La topologia e costi noti a ciascun nodo, l'albero del percorso a costo minimo viene calcolato utilizzando l'algoritmo di **Dijkstra** e memorizzato nel cosiddetto "database di stato dei collegamenti", distribuito a tutti i router periodicamente. Gli aggiornamenti, invece, vengono trasmessi in broadcast ai router dell'intero AS (**flooding**) direttamente su **IP**. Le funzionalità principali sono:

- **Sicurezza:** autenticazione dei messaggi OSPF con algoritmi di crittografia.
- **Percorsi multipli con costo uguale:** possibilità di utilizzare percorsi multipli per instradare il traffico.
- **Supporto integrato per il routing unicast e multicast:** multicast OPSF (**MOSPF**) utilizza lo stesso database di collegamenti di OSPF.
- **Struttura gerarchica degli AS:** possibilità di strutturare grandi domini di routing in gerarchie di AS.

4.7.3 Struttura di un messaggio OSPF

Formato da un header in comune e un body che dipende dal tipo.

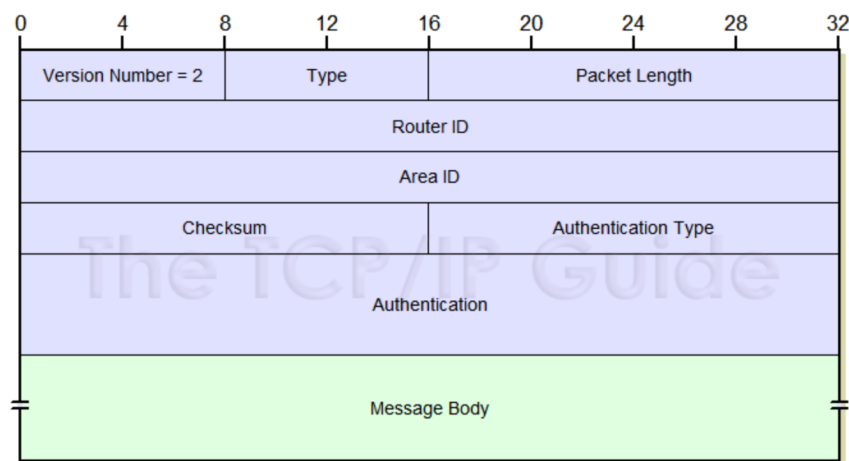


Figura 4.9: Messaggio con OSPF.

L'header è formato da:

- **Versione number.**
- **Tipo:**
 - Hello.
 - DB Description.
 - Link State Req.
 - Link State Update.
 - Link State ACK.
- **Dimensione:** misurata in byte (header + body).
- **Router ID:** ip dell'interfaccia da dove è inviato il messaggio.
- **AreaID:** area a cui il messaggio appartiene.
- **Checksum:** include tutto tranne che i campi di autenticazione.
- **Tipo di autenticazione:** può essere 0 (niente), 1 (con password) e 2 (crittografata).
- **Informazioni di autenticazione.**

4.7.4 Struttura della gerarchia OSPF

Consente di suddividere i grandi sistemi autonomi (AS) in aree in modo da avere una gerarchia interna a due livelli:

- **Aree locali:** comunicano le rotte verso altre aree locali del sistema autonomo ai router di tali aree.
- **Aree Backbone:** instrada il traffico tra le aree del sistema autonomo, contiene tutti i border router.

L'algoritmo Link State viene utilizzato all'interno di ciascuna area.

4.8 IPv6

Capitolo 5

Livello Trasporto

5.1 Introduzione

Il livello di trasporto estende il servizio di consegna del protocollo IP tra due host terminali a un servizio di consegna per due processi applicativi in esecuzione sugli host. Inoltre, aggiunge alcune funzionalità, come il controllo di errori e il multiplexing/demultiplexing dei messaggi fra processi. I protocolli principali sono due:

1. **TCP (Transmission Control Protocol).**
2. **UDP (User Datagram Protocol).**

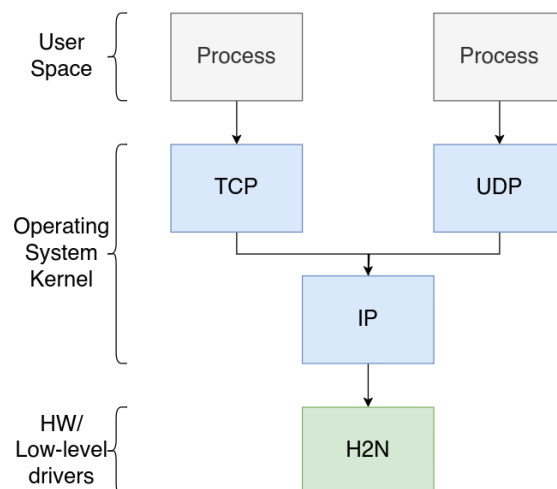


Figura 5.1: Schema comunicazione con protocolli e livelli.

5.1.1 Multiplexing e Demultiplexing

Il protocollo IP non trasferisce dati tra i processi applicativi in esecuzione sui nodi finali, questo è compito dei protocolli di trasporto:

- **Multiplexing:** creazione di segmenti derivanti da messaggi di diversi processi applicativi, avviene sul nodo sorgente.
- **Demultiplexing:** ogni segmento del livello di trasporto ha un campo contenente le informazioni utilizzate per determinare a quale processo il segmento deve essere consegnato, avviene al nodo di destinazione.

I protocolli UDP e TCP aggiungono per fare questo due campi nell'header del segmento:

1. **Numero di porta del mandante.**
2. **Numero di porta del destinatario.**

Porta: strumento utilizzato per il multiplexing/demultiplexing per identificare in modo **univoco** i **processi** e che permette di eseguire connessioni multiple sullo stesso host. Le porte si dividono in tre categorie:

1. **Porte Well-Known:** da 0 a 1023.
2. **Porte Registerate:** da 1024 a 49151.
3. **Porte Dinamiche o Private:** da 49152 a 65535.

Due processi client, che comunicano con lo stesso servizio applicativo, sono sempre distinti in base alla tupla:

$[(\text{Source IP}, \text{Source Port}), (\text{Destination IP}, \text{Destination Port})]$

5.2 Protocollo UDP

UDP (User Datagram Protocol) è un protocollo di trasporto molto leggero, partendo dall'**IP Datagram**, aggiunge alcuni header:

Sender Port	Destination Port
Length	Checksum
Message Payload	

Figura 5.2: Segmento UDP.

In particolare:

- **Numero di porta sorgente** (16 bit).
- **Numero di porta di destinazione** (16 bit).
- **Lunghezza** (16 bit): header + data, dove l'header è sempre 8 byte.
- **Checksum** (16 bit).

5.2.1 Campo Checksum

Questo campo è stato aggiunto perchè non tutte le connessioni a livello H2N hanno un controllo dell'errore e inoltre, il controllo dell'errore con checksum a livello IP controlla solo il datagram IP. Quindi, a livello di trasporto, non ci sarebbe stato nessun modo di avere un ripristino dall'errore.

Nel pratico:

- **Sorgente:** tratta il contenuto del segmento come una sequenza di numeri interi a 16 bit, somma il contenuto dei segmenti in complemento a uno e invia il valore del checksum nel campo del segmento UDP.
- **Destinatario:** calcola il checksum del segmento ricevuto e verifica se il valore del checksum calcolato è uguale a quello ricevuto.

ESEMPIO:

Word 1	0110011001100110
Word 2	0101010101010101
Word 3	0000111100001111
Somma	1100101011001010
Complemento a 1	0011010100110101

Il destinatario calcola il proprio checksum del segmento UDP, senza calcolare il complemento a uno:

- **OK:** checksum destinazione + checksum UDP = 1111111111111111.
- **ERRORE:** Checksum destinazione + checksum UDP != 1111111111111111.

5.3 Protocollo TCP

Il protocollo TCP fornisce un livello di trasporto affidabile e orientato alla connessione. Ha alcuni servizi aggiuntivi rispetto a UDP:

- **Orientato alla connessione:** include le fasi di stabilimento, utilizzo e chiusura della connessione.
- **Orientato al flusso di dati:** considera il flusso di dati dall'host mittente al destinatario.
- **Trasferimento con buffer:** i dati vengono memorizzati in un buffer e quindi inseriti in un pacchetto quando il buffer è pieno.
- **Connessione full-duplex (bidirezionale):** una volta stabilita la connessione, è possibile il trasferimento simultaneo in entrambe le direzioni della connessione.

5.3.1 Affidabilità

Affidabile significa che TCP gestisce il trasferimento di un flusso di dati con la sequenza corretta (ACK + timeout + ritrasmissione):

- Ogni trasmissione riuscita viene notificata (**ACK**) dall'host ricevente.
- Se l'host mittente non riceve una conferma entro un intervallo di tempo predefinito (**timeout**), il mittente ritrasmette i dati.
- Gli **ACK** e le **ritrasmissioni** dovute a eventuali perdite vengono gestite in modo trasparente rispetto al processo applicativo.

5.3.2 Trasferimento con buffer

Per risolvere molti dei problemi evidenziati sopra, il livello TCP deve necessariamente utilizzare un buffer per evitare:

- Invio asincrono dei dati rispetto al processo applicativo.
- Tempi di trasmissione differenti.
- Capacità di invio e ricezione differenti.
- Segmenti persi o fuori sequenza.

Inoltre, il protocollo TCP ha altri due compiti fondamentali:

1. **Controllo della congestione:** l'host mittente deve ridurre la velocità di trasmissione dei pacchetti quando la rete è congestionata.
2. **Controllo del flusso:** l'host mittente non deve sovraccaricare l'host ricevente.

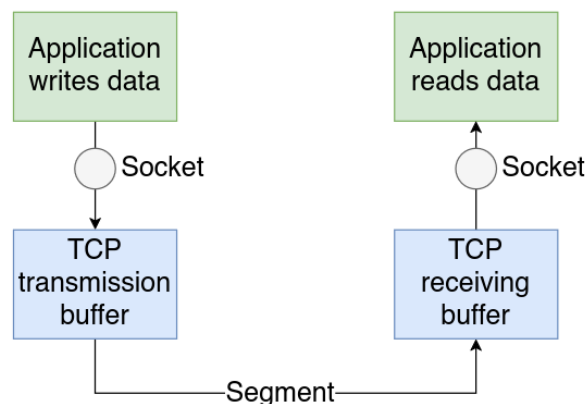


Figura 5.3: Esempio di utilizzo del buffer TCP.

Un'applicazione può anche indicare a TCP di inviare i dati presenti nel buffer, senza attendere che quest'ultimo sia pieno.

5.4 TCP Segment

L'insieme di dati che il livello TCP richiede di trasferire al livello IP è chiamato segmento TCP ed è formato da una parte di header e da una parte di byte di dati.

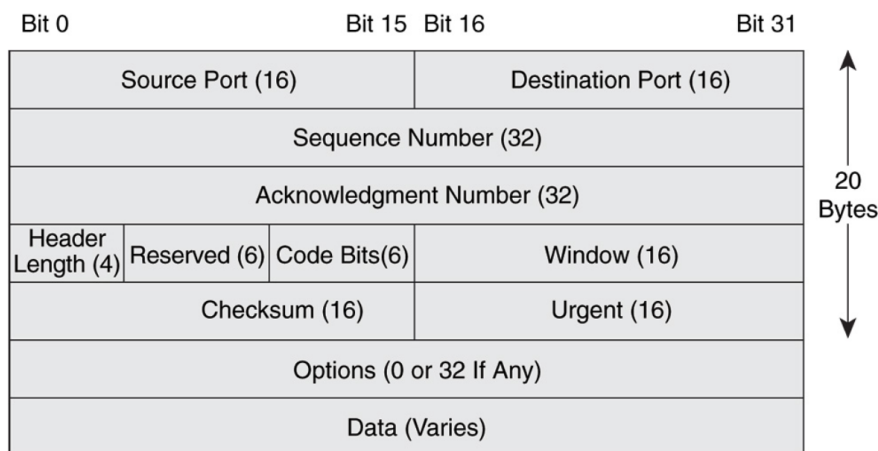


Figura 5.4: Esempio di utilizzo del buffer TCP.

Nello specifico:

- **Porta sorgente** (16 bit).
- **Porta di destinazione** (16 bit).
- **Sequence number** (32 bit): sequence number relativo al flusso di byte trasmesso.
- **Acknowledgment number** (32 bit): ACK relativo al sequence number del flusso di byte ricevuto.
- **Lunghezza dell'header** (4 bit): lunghezze in multipli di 32 bit dell'header, se non ha opzioni è sempre 20 byte.
- **Reserved** (6 bit): riservato per usi futuri.
- **Code bit** (6 bit): usato per le varie flag:
 - URG: dati segnati come urgenti dal layer applicativo.
 - ACK: valore dell'ACK è valido.
 - PSH: i dati devono essere passati all'applicazione subito.
 - SYN, FIN, RST: usati per stabilire, chiudere o interrompere una connessione.
- **Dimensione della finestra** (16 bit): indica il numero di byte che il destinatario è disposto ad accettare.
- **Checksum** (16 bit): controllo dell'integrità come nell'UDP.
- **Urgent** (16 bit): puntatore alla fine dei dati urgenti.
- **Opzioni** (0-32 bit): campi opzionali di varie dimensioni (come MSS - maximum segment size).
- **Padding di zeri** se necessario.

Alcuni esempi di dati urgenti posso essere i segnali come *Ctrl + Z* o *Ctrl + C*, in modo che possano arrivare immediatamente. Quando vengono ricevuti, saltano direttamente lo stream di byte e arrivano all'applicazione. Sono posizionati all'inizio del segmento, per questo il puntatore Urgent punta alla loro fine.

5.5 Inizializzare e chiudere una connessione TCP

Nel protocollo TCP, il mittente e il destinatario stabiliscono la connessione e inizializzano le variabili dei segmenti prima di iniziare il trasferimento, utilizzando un modello **client/server**.

Per stabilire una connessione il client invia un segmento particolare, dove imposta il campo **SYN** a 1, al server. Deve contenere:

- La porta del server.
- L'**ISN (Initial Sequence Number)** del client, un numero a 32 bit pseudo randomico, da cui inizierà la numerazione.
- L'**MRW (Maximum Receive Window)**, ossia il numero di byte che il client può ricevere nel buffer.
- L'**MSS (Maximum Segment Size)**, la grandezza massima del segmento, anche se non sempre è inviata.

Questo segmento TCP non contiene nessun dato, ma solo l'header. Il server, al sua volta, risponderà con un segmento con **SYN** a 1 e con gli stessi campi, ma con i valori riferiti a lui e con un **ACK** con valore dell'**ISN** del client aumentato di uno. Infine, il client risponderà con un **ACK** del valore dell'**ISN** del server aumentato di uno.

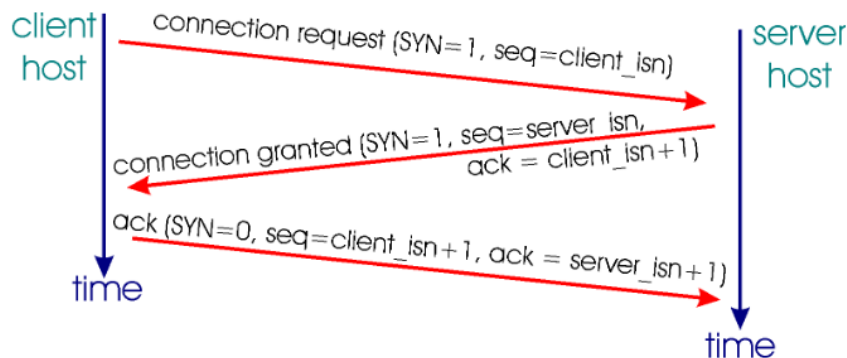


Figura 5.5: Esempio di three-way-handshaking.

Per chiudere una connessione in modo pulito, il client deve mandare un segmento con il campo **FIN** a 1 al server, il quale dopo aver risposto al client con un **ACK**, chiude la connessione lato client e invia a sua volta un segmento con campo **FIN** a 1 al client. Infine, il client dopo aver ricevuto il **FIN** del server gli invia un **ACK**. Il server appena riceve l'**ACK** chiude la connessione, il client la chiude dopo un tempo di timeout. Un altro modo per interrompere una connessione è impostare il campo **RST** a 1, questo farà sì che l'altro nodo chiuda immediatamente la connessione e che tutte le risorse vengano liberate.

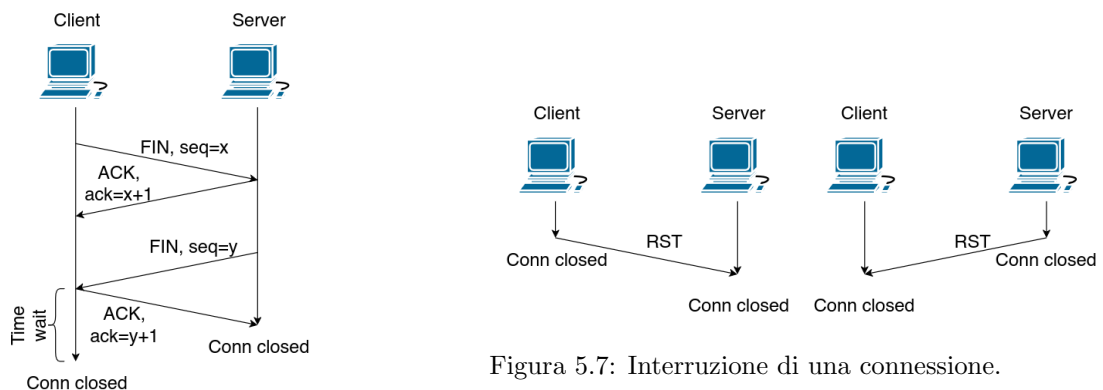


Figura 5.7: Interruzione di una connessione.

Figura 5.6: Chiusura pulita di una connessione.

5.6 Affidabilità nel protocollo TCP

Ci sono diversi meccanismi per garantire affidabilità in modo trasparente rispetto al processo applicativo:

- **ACK.**
- **Timeout.**
- **Ritrasmissione.**

Ogni trasmissione riuscita viene **confermata** dall'host ricevente. Se l'host mittente non riceve una conferma entro il **timeout**, il mittente ritrasmette i **dati**.

5.6.1 Stima del Timeout

La stima del timeout viene fatto usando l'**RTT (Round Trip Time)**. Il valore è molto importante calcolarlo bene, perchè se troppo basso si rischia di dover inviare il segmento più volte, se troppo alto, invece, si ha una lenta risposta ai segmenti persi. In generale, il timeout deve essere dell'RTT. Ci sono due modi per misurare il Round Trip Time:

1. **SampleRTT**: misura il tempo dall'invio del segmento alla ricezione dell'ACK, varia più volte dinamicamente.
2. **EstimatedRTT**: media pesata per sapere l'RTT a un tempo **t**:

$$\text{EstimatedRTT}(t) = (1 - \alpha) \cdot \text{EstimatedRTT}(t - 1) + \alpha \cdot \text{SampleRTT}(t)$$

Dove, dato un numero n di casi:

$$\alpha = \frac{1}{n + 1}$$

Attualmente, il **timeout** è calcolato usando l'EstimatedRTT e un piccolo margine:

$$\text{Timeout}(t) = \text{EstimatedRTT}(t) + \text{Dev}(t)$$

Dove:

$$\text{Dev}(t) = (1 - \alpha)\text{Dev}(t - 1) + \alpha \cdot |\text{SampleRTT}(t) - \text{EstimatedRTT}(t)|$$

5.7 Gestione del Buffer TCP

TCP, essendo parte del Sistema Operativo e non del layer applicativo, deve riuscire a gestire tutte gli eventi asincroni che accadono. Infatti, il protocollo TCP non sa quando un processo vorrà inviare i dati o quando li vorrà ricevere. Per questo si inseriscono temporaneamente i dati dentro un **buffer**.

Alcune definizioni:

- **RTT**: tempo di un segmento per arrivare a destinazione e tornare indietro.
- **Tempo di propagazione (T_p)**: $\frac{RTT}{2}$.
- **Utilizzo (U)**: percentuale di utilizzo delle risorse.
- **Frequenza di Trasmissione (T_r)**.
- **Packet Length: $L(pkt)$** .
- **Tempo di trasmissione di un pacchetto (TT)**: $\frac{L(pkt)}{T_r}$.
- **Tempo di trasferimento di un segmento(T)**: $T = \lceil \frac{RTT}{2} + L(pkt) \rceil + \frac{RTT}{2}$

I buffer servono per passare da un meccanismo **Stop-and-Wait** a uno in **pipeline**. Devono essere implementati sia dal lato del mandante che del destinatario, insieme a protocolli **Sliding Windows** per sapere quanti pacchetti mandare senza ricevere il loro ACK.

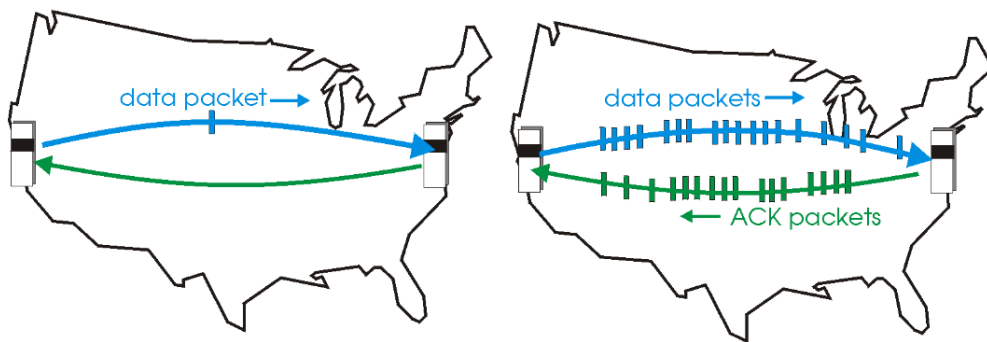


Figura 5.8: Protocolli Stop-and-Wait e con Pipeline.

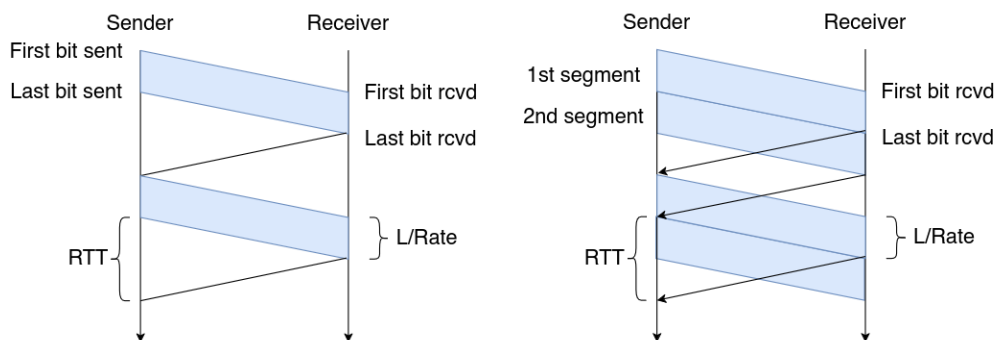


Figura 5.9: Protocolli Stop-and-Wait e con Pipeline.

5.7.1 Sliding Windows

Il mittente assegna a ciascun segmento un numero di sequenza **segment_no** nell'intervallo $[0, 2^{32} - 1]$.

Mandante

In ogni momento, il mittente mantiene una sliding window sugli indici di segmento e solo quelli all'interno della finestra possono essere trasmessi. La dimensione della finestra utile del mittente è controllata dal destinatario. Usa tre variabili:

- **Sender Window Size (SWS):** il massimo di segmenti che può mandare senza ricevere un ACK.
- **Last ACK Received (LAR):** numero di sequenza dell'ultimo ACK ricevuto.
- **Last Segment Send (LSS):** numero di sequenza dell'ultimo segmento inviato.

NB:

$$LAS - LSR \geq RWS$$

Ricevente

In qualsiasi momento, il destinatario mantiene una sliding window sugli indici dei segmenti ricevuti. La dimensione della finestra e le modalità di gestione dipendono dall'algoritmo utilizzato. Usa tre variabili:

- **Receive Window Size (RWS):** indica il numero massimo di segmenti fuori sequenza che può accettare.
- **Largest Acceptable Segment (LAS):** numero di sequenza più grande che può accettare.
- **Last Segment Received (LSR):** numero di sequenza dell'ultimo segmentoricevuto in ordine.

NB:

$$LSS - LAR \geq SWS$$

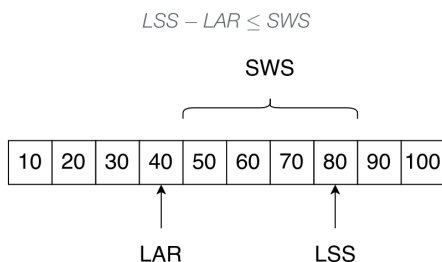


Figura 5.10: Chiusura pulita di una connessione.

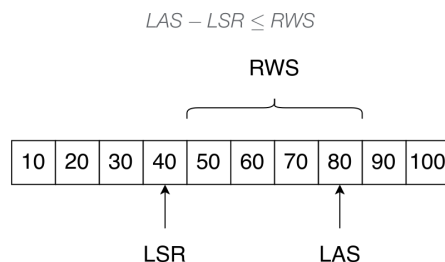


Figura 5.11: Interruzione di una connessione.

5.7.2 Flow Control

L'obiettivo è non far inviare al mandante più dati di quelli che il buffer del destinatario può contenere. Il ricevitore comunica esplicitamente al mittente la quantità di spazio libero nel buffer di ricezione TCP e varia dinamicamente, quindi ogni segmento ACK comunica al mittente il numero massimo di byte ricevibili.

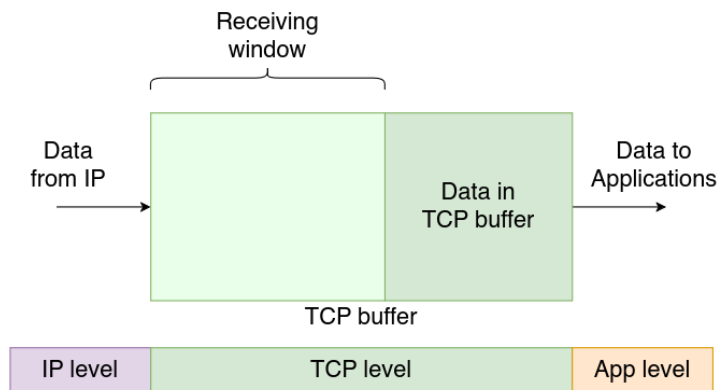


Figura 5.12: Buffer TCP.

L'ACK contiene due informazioni:

1. Quanti byte ha ricevuto il recipiente.
2. Quanti byte può ancora ricevere in quel momento.

$$\text{Advertised Window} = \text{MaxRcvBuffer} - ((\text{NextByteExpected} - 1) - \text{LastByteRead})$$

$$\text{EffectiveWindow} = \text{AdvertisedWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

Quando il mandante riceve **Advertised Window** = 0, teoricamente non può più mandare dati e per questo non riesce a capire quando può ricominciare a mandare dati. La soluzione nel protocollo TCP è mandare costantemente mandare un segmento con un byte di dato.

5.7.3 Congestion Control

La congestione in una rete può causare diversi problemi, come la perdita di pacchetti oppure tempi di delay molto elevati. Ci sono due modi per approcciarsi alla gestione della congestione:

1. **Controllo end-to-end:** approccio usato dal protocollo TCP, non viene gestito dalla rete, ma si guardano i pacchetti persi e il tempo di delay.
2. **Controllo di rete:** i router dicono ai nodi terminali lo stato della congestione della rete.

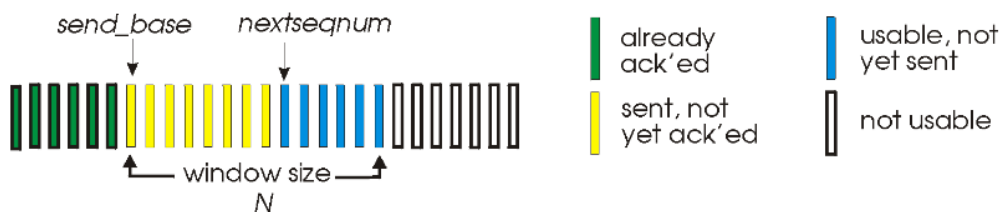


Figura 5.13: Dimensione della finestra nel TCP.

Le prestazioni nel protocollo TCP vengono misurate tramite il **Throughput**:

$$\text{Throughput} = \frac{w \cdot MSS}{RTT}$$

Dove w è il numero di segmenti della finestra. Esistono diversi algoritmi per evitare la congestione della rete:

Slow Start

Inizia con la dimensione della finestra di congestione pari a 1 e poi in modo esponenziale aumenta fino al threshold.

```
for each ACK segment do
    CW++
    if timeout OR CW > threshold then
        break
```

Tahoe Algorithm

Cresce in modo esponenziale fino al threshold, poi incrementa di 1.

{Slow start ended, $CW \geq \text{threshold}$ }

```
if not loss then
    for all ACK segment
        CW++
else
    threshold = CW/2
    CW = 1
    do slow start
```

Reno Algorithm

Controlla due tipi di errore:

1. **Timeout:** riduce la finestra di congestione a 1.
2. **Tre ACK duplicati:** riduce la finestra di metà.

{Slow start ended, $CW \geq \text{threshold}$ }

```

if not loss then
    for all ACK segment
        CW++
else
    threshold = CW/2
    if Loss detected by timeout then
        CW = 1
        do slow start
    if loss detected by triple ACK then
        CW = CW/2

```

Reno, in confronto a Tahoe, ha un recupero molto più veloce, ma ormai, entrambi sono obsoleti.

Vegas Algorithm

Il timeout è basato sull'RTT e ha un contatore che controlla ogni pacchetto.

```

x(t) = [WSize(t) - WSize(t-1)] / [RTT(t) - RTT(t-1)]
if x(t) > 0
    CW = CW - CW/8
else
    CW = CW + MSS

```

Se aumenta la congestione aumenta pure l'RTT e viceversa. La finestra di congestione cambia ogni $2 \cdot \text{RTT}$.

BIC (Binary Increase Congestion)

Mantiene 4 valori diversi di finestre di congestione:

1. **Attuale.**
2. **Massima.**
3. **Minima.**
4. **Prevista (valore fra massimo e minimo).**

Nello specifico:

1. Quando la differenza tra la finestra minima e quella massima è ampia, si ricorre a una crescita aggressiva della finestra (**incremento additivo**).
2. Quando la finestra corrente raggiunge quella prevista senza errori, la finestra minima diventa quella prevista.
3. In caso di errore, la finestra massima assume il valore della finestra corrente. La finestra corrente si riduce al nuovo valore minimo.
4. Man mano che le finestre minime e massime si restringono, la ricerca diventa meno aggressiva.

CUBIC

Evoluzione di BIC, mantiene la sua stabilità e scalabilità, ma aumenta la fairness. Infatti, la dimensione della finestra di congestione dipende la tempo e non dall'RTT.

5.7.4 Buffer Bloating

Problema regolato dalla presenza di buffer molto grandi nella rete, tipicamente di dimensioni variabili. Si verifica in presenza di trasferimenti di dati di lunga durata e dalla presenza di più flussi TCP simultanei. Quando i buffer sono pieni, l'RTT è così elevato che il timeout per la perdita di pacchetti rallenta l'intervento del controllo della congestione. Il risultato è il calo del throughput a causa dei ritardi e un elevato tasso di perdita di pacchetti. Esistono alcune proposte per risolvere questo problema come:

- **CoDel Algoritmo (Controlled Delay)** come algoritmo di AQM (Adaptive Queue Management) per gestire la grandezza dei buffer in modo dinamico per più flussi TCP.
- **Google SPDY.**
- **HTTP/2.**

Capitolo 6

Socket Programming

6.1 API (Application Program Interface)

API: meccanismo utilizzato come ponte tra il sistema operativo e l'applicazione di rete che può essere utilizzato per sviluppare applicazioni. Fa parte del sistema operativo e consente ai processi applicativi di utilizzare protocolli di rete, definendo:

- Le operazioni consentite.
- Gli argomenti per ciascuna operazione.

L'**API** per **Socket**, inizialmente definita per **BSD Unix** per utilizzare i protocolli **TCP/IP**, è disponibile su vari sistemi operativi.

6.1.1 Socket

Socket: sono **API** che consentono ai programmatori di gestire le comunicazioni tra **processi**. Consentono la comunicazione tra processi che possono risiedere su **host** diversi e costituiscono quindi lo strumento di base per la creazione di servizi di rete. In pratica, permettono a un programmatore di effettuare trasmissioni **TCP** e **UDP** senza doversi preoccupare dei dettagli di livello inferiore, che sono gli stessi per ogni comunicazione. Forniscono inoltre un'interfaccia diretta al livello di rete, le **socket raw**. Vengono utilizzati in un ambiente **Linux/Unix** in modo simile ai file per le operazioni di **I/O**.

Forniscono una serie di funzioni:

- **socket():** crea un punto di comunicazione.
- **bind():** fissa un indirizzo locale alla socket.
- **listen():** si mette a disposizione per accettare richieste.
- **accept():** aspetta una richiesta di connessione.
- **write():** manda i dati tramite la connessione.
- **read():** riceve i dati dalla connessione.
- **close():** chiude la connessione.

6.2 Creare e gestire una connessione

Inizialmente, va dichiarato al sistema operativo dell'intenzione di stabilire una **connessione** con specifica delle caratteristiche. Successivamente, si possono aprire le connessioni (diverse tra lato **server** e lato **client**):

- Il server presume di definire la connessione prima del client e attende che il client si connetta alla porta specificata.

- Il client presume che il server sia già attivo e tenta di connettersi specificando l'indirizzo e la porta del server.

Lo scambio di dati è **bidirezionale** (trasmissione e ricezione). Alla fine della comunicazione, la connessione viene chiusa.

Creazione del socket:

```
descriptor = socket(protofamily, type, protocol)
```

Dove:

- **Descriptor:** handler del socket, int.
- **Protocol Family:** specifica il dominio di comunicazione:
 - **AF_INET:** protocolli Internet IPv4.
 - **AF_INET6:** protocolli Internet IPv6.
 - **AF_UNIX:** comunicazioni locali (stessa macchina).
 - **AF_PACKET:** accesso diretto al livello di collegamento.
- **Socket Type:** specifica la semantica della comunicazione:
 - **SOCK_STREAM:** connessione TCP.
 - **SOCK_DGRAM:** connessione UDP.
 - **SOCK_RAW:** accesso grezzo al protocollo.

Non tutte e le combinazioni sono valide.

Binding del socket:

```
bind(socket, localaddr, addrlen)
```

Dove:

- **Socket:** descriptor del socket.
- **Local Address:** indirizzo su cui mettersi in ascolto.
- **Address Len:** lunghezza dell'indirizzo.

Mettersi in ascolto sul socket:

```
listen(socket, queuesize)
```

Dove:

- **Socket:** descriptor del socket.
- **Queue Size:** numero di richieste in sospeso consentite.

Accettare nuove connessioni:

```
newsock = accept(socket, caddress, caddresslen)
```

Dove:

- **Socket:** descriptor del socket.

- **Client address:** puntatore all'indirizzo del client.
- **Dimensione dell'indirizzo:** puntatore a un intero con la dimensione dell'indirizzo del client.
- **New Socket:** descriptor del nuovo socket creato.

Creare una nuova connessione:

```
connect(socket, saddress, saddresslen)
```

Dove:

- **Socket:** descriptor del socket.
- **Server address:** puntatore all'indirizzo del client.
- **Dimensione dell'indirizzo:** puntatore a un intero con la dimensione dell'indirizzo del client.

Inviare dati:

```
send(socket, data, length, flags)
```

Dove:

- **Socket:** descriptor del socket.
- **Data:** puntatore ai dati da inviare.
- **Dimensione dei dati:** pintero con la dimensione dei dati.
- **Flags:** flag da specificare nel caso.

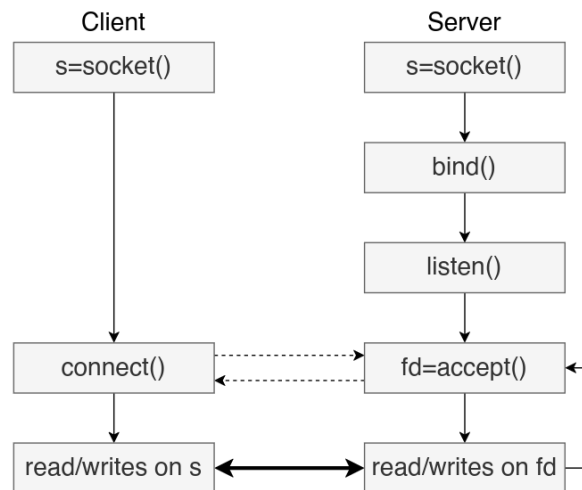


Figura 6.1: Connessione e comunicazione con socket.

Per le connessioni UDP, a differenza delle connessioni TCP, non c'è bisogno di mettersi in ascolto. Si possono mandare dati e ricevere dati direttamente sulla rete, usando:

```
recvfrom(sockfd, *buf, size, flags, *src_addr, *addrlen)
sendto(sockfd, *buf, size, flags, *dst_addr, addrlen)
```

Queste funzioni sostituiscono le syscall `write()` e `read()`.

6.2.1 Codice C Server Side:

```
* A simple server using TCP
The port number is passed as an argument */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>

void error(char *msg){
    perror(msg);
    exit(1);
}

int main(int argc, char *argv[]){
    int sockfd, newsockfd, portno, clilen;
    char buffer[256];
    struct sockaddr_in serv_addr, cli_addr;
    int n;

    if (argc < 2) {
        fprintf(stderr, "ERROR, no port provided\n");
        exit(1);
    }

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0)
        error("ERROR opening socket");

    bzero((char *) &serv_addr, sizeof(serv_addr));
    portno = atoi(argv[1]);
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;
    serv_addr.sin_port = htons(portno);

    if (bind(sockfd, (struct sockaddr *) &serv_addr, sizeof(serv_addr)) < 0)
        error("ERROR on binding");

    listen(sockfd, 5);
    clilen = sizeof(cli_addr);
    newsockfd = accept(sockfd, (struct sockaddr *) &cli_addr, &clilen);
    if (newsockfd < 0)
        error("ERROR on accept");

    bzero(buffer, 256);
    n = read(newsockfd, buffer, 255);
    if (n < 0)
        error("ERROR reading from socket");

    printf("Here is the message: %s\n", buffer);
    n = write(newsockfd, "I got your message", 18);
    if (n < 0)
        error("ERROR writing to socket");

    return 0;
}
```

6.2.2 Codice C Client Side:

```

include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

void error(char *msg){
    perror(msg);
    exit(0);
}

int main(int argc, char *argv[]){
    int sockfd, portno, n;
    struct sockaddr_in serv_addr;
    struct hostent *server;
    char buffer[256];

    if (argc < 3) {
        fprintf(stderr, "usage %s hostname port\n", argv[0]);
        exit(0);
    }

    portno = atoi(argv[2]);
    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd < 0)
        error("ERROR opening socket");

    server = gethostbyname(argv[1]);
    if (server == NULL) {
        fprintf(stderr, "ERROR, no such host\n");
        exit(0);
    }

    bzero((char *) &serv_addr, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    bcopy((char *)server->h_addr, (char *)&serv_addr.sin_addr.s_addr, server->
        h_length);
    serv_addr.sin_port = htons(portno);

    if (connect(sockfd, (const struct sockaddr *) &serv_addr, sizeof(serv_addr)) <
        0)
        error("ERROR connecting");

    printf("Please enter the message: ");
    bzero(buffer, 256);
    fgets(buffer, 255, stdin);
    n = write(sockfd, buffer, strlen(buffer));
    if (n < 0)
        error("ERROR writing to socket");

    bzero(buffer, 256);
    n = read(sockfd, buffer, 255);
    if (n < 0)
        error("ERROR reading from socket");
    printf("%s\n", buffer);

    return 0;
}

```

6.2.3 Codice Python Server Side:

```
#!/usr/bin/env python3
import socket
import sys
import time

HOST = '127.0.0.1' # Standard loopback interface address (localhost)
PORT = int(sys.argv[1]) # Port to listen on

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    conn, addr = s.accept()
    #print('Connected by', addr)
    data = conn.recv(1024)
    print('Here is the message: %s'% data.decode('utf-8'))
    conn.sendall('I got your message'.encode('utf-8'))
    # socket must be closed by client! sleep for 1 second
    time.sleep(1) # otherwise socket goes to TIME_WAIT!
```

6.2.4 Codice Python Client Side:

```
#!/usr/bin/env python3
import socket
import sys
HOST=sys.argv[1]
PORT=int(sys.argv[2])
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    msg = input('Please enter the message: ')
    s.sendall(msg.encode('utf-8'))
    data = s.recv(1024)
    s.close()
    print('Received: %s'% data.decode('utf-8'))
```

Capitolo 7

DNS (Domain Name System)

7.1 Definizione:

DNS: un'applicazione Internet che fornisce servizi ad altre applicazioni Internet. Introduce un nuovo livello di denominazione per le risorse Internet (oltre agli indirizzi IP) progettato per essere più intuitivo. È un esempio di sistema distribuito su scala globale che funziona molto bene, ha superato di gran lunga le aspettative in termini di scalabilità e ha superato la prova del tempo.

7.2 Host identifiers

Ogni host in un rete necessita di almeno un IP, ma può avere anche più **hostname**.

Hostname: è una stringa **alfanumerica** di **255** caratteri, formata da **label**, ognuno divisa da un punto. Molto più facile da ricordare rispetto a un indirizzo IP e comoda per ambienti dinamici, infatti, si può usare lo stesso hostname anche quando l'ip cambia.

Il vero hostname è chiamato **canonico**, da cui si possono derivare degli **alias**.

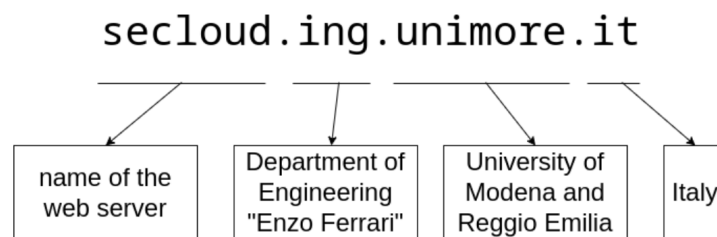


Figura 7.1: Esempio di Hostname canonico.

Ogni label rappresenta un livello di gerarchia diverso. Ovviamente, implementando un sistema di questo tipo, deve essere anche creato un modo per convertire gli hostname nel loro IP e viceversa.

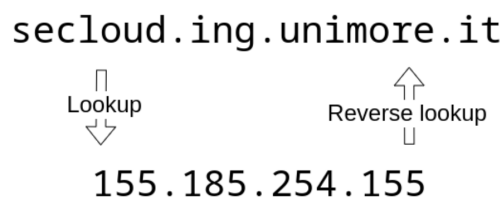


Figura 7.2: Lookup e reverse lookup.

All'inizio, venivano usati dei file contenenti ogni IP col proprio hostname, ma col crescere dei file si è cercato un modo più scalabile e dinamico, il DNS.

7.3 Name Servers

Divisi in diversi livelli:

- **Root name server:** 13 server, denominati dalla A alla M. Sono distribuiti in ambienti altamente controllati e protetti e gestiti da 12 organizzazioni indipendenti. Inoltre, sono **autorevole**, devono sapere tutti gli IP dei TLD name server.
- **TLD (Top Level Domain) name server:** deve fornire almeno due name server diversi che siano autorevoli per la sua zona. Tutti conoscono gli IP dei SLD name server.
- **SLD (Second Level Domain) name server:** deve fornire almeno due name server diversi che siano autorevoli per la sua zona. Conoscono tutti gli IP dei name server più sotto.
- **Local name server.**

7.3.1 Resource Records

In un nameserver, i dati vengono memorizzati come un insieme di **resources records**. I nameserver consentono meccanismi di ricerca IP diretti e inversi. Ogni record di risorse ha 6 campi:

Field	Description	Length (octets)
NAME	Name of the node to which this record pertains	Variable
TYPE	Type of RR in numeric form (e.g., 15 for MX RRs)	2
CLASS	Class code	2
TTL	Count of seconds that the RR stays valid (The maximum is $2^{31}-1$, which is about 68 years)	4
RDLENGTH	Length of RDATA field (specified in octets)	2
RDATA	Additional RR-specific data	Variable, as per RDLENGTH

Figura 7.3: Campi di un resource record.

Alcuni record comuni sono:

- **A:** contiene un indirizzo ipv4 associato a un Fully Qualified Domain Name (FQDN).
- **AAAA:** contiene un indirizzo ipv6 associato a un Fully Qualified Domain Name (FQDN).
- **NS:** memorizza il FQDN del name server autorevole per una determinata zona.
- **NX:** memorizza il FQDN dei server di posta che riceveranno le email per i destinatari di un determinato dominio.
- **CNAME:** contiene il nome canonico di un determinati alias.
- **PRT:** contiene il CNAME associato a un determinati IP.
- **SOA:** contiene parametri di configurazione per la zona.
- **TXT:** associa a una FQDN una stringa arbitraria.

Si possono vedere direttamente le query con il comando **dig**:

```
dig [RR_type] FQDN [@nameserver]
```

Per esempio:

```
dig A www.unimore.it @8.8.8.8
dig MX unimore.it @155.185.1.2
dig SOA unimore.it @155.155.1.2
dig NS unimore.it @193.0.14.129
```

7.3.2 Risoluzione dei nomi in modo distribuito

Nessun server DNS conosce tutti i valori di tutti i **record RR** dell'intero Internet, è necessaria una qualche forma di **cooperazione**. Il DNS utilizza un approccio client-server:

1. Il client avvia il processo di ricerca inviando una query a un name server, di solito si tratta di quello locale configurato in ciascun dispositivo.
2. Se conosce la risposta, la fornisce direttamente al client.
3. Se non conosce la risposta deve chiederla a un altro name server.

Le query possono essere **ricorsive**, dove ogni name server chiede in automatico a un altro name server, oppure **iterative** dove sarà l'utente a interrogare più name server. Ogni name server possiede una cache con alcune risposte, alla scadenza del loro **TTL** verranno eliminate.

7.4 Pacchetto DNS

Facendo parte del livello applicativo, oltre agli header del protocollo IP e del protocollo UDP, al pacchetto viene aggiunto quello del protocollo DNS.

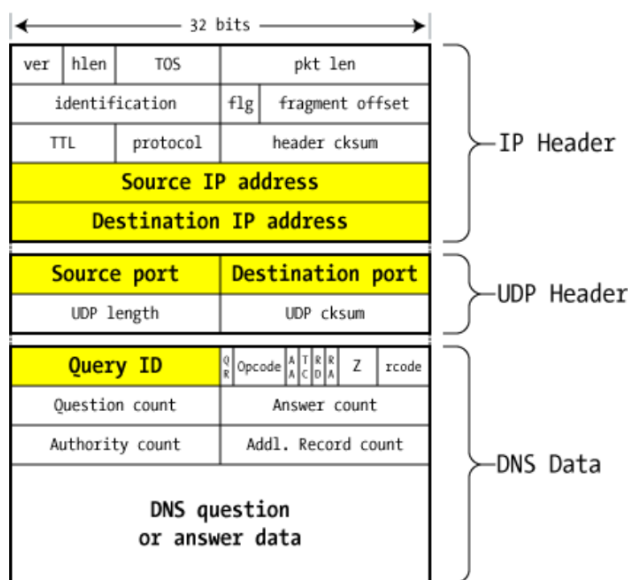


Figura 7.4: Campi di un resource record.

In particolare:

- **Query ID:** identificatore univoco creato nel pacchetto di query e ripetuto dal server nella risposta.
- **QR (Query / Response):** 0 se è una query, 1 se è una risposta.
- **Opcode:** per il tipo di query, standard è 0.
- **AA (Authorative Answer):** 1 se una risposta è autorevole.
- **TC (Trunkated):** la risposta non stava nel pacchetto UDP, sarà necessario fare una nuova richiesta con un segmento TCP.
- **RD (Recursion Desired):** 1 se si vuole fare una query ricorsiva, 0 altrimenti.
- **RA (Recursion Available):** 1 se si può fare una query ricorsiva, 0 altrimenti.
- **Z:** non usato, deve essere 0.
- **rcode:** codice di risposta del server, successo o fallimento.
- **Question record count.**
- **Answer/Authorative/Additional record count:** inseriti dal server.
- **NDS question:** campo che contiene i dati della richiesta/risposta.

Capitolo 8

Esercizi

8.1 Creare un rete

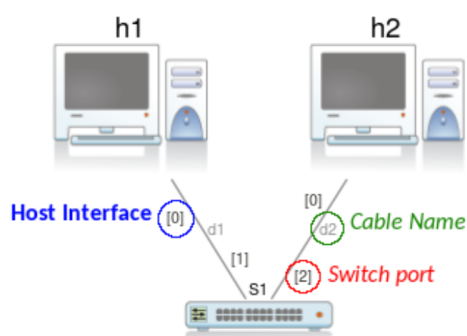


Figura 8.1: Esempio di una rete in Marionet.

In Marionet ci sono due tipologie di cavi:

1. **Cavi crossover:** per connettere macchine sullo stesso livello dello stack.
2. **Cavi normali:** per connettere macchine su livelli diversi dello stack.